User-Centered Design and Development of a System for Management of Module Handbooks

Bachelor Thesis

submitted as part of the Bachelor's degree program
Media Informatics
at Berliner Hochschule für Technik

Submitted by: Yichen Zhong

Student ID: 104636

Submission date: 01 October 2026

First Supervisor: Prof. Dr. Zbigniew Jerzak

Second Supervisor: Prof. Dr. Selcan Ipek-Ugay

Berliner Hochschule für Technik

Abstract

Module handbooks at Berliner Hochschule für Technik (BHT) are currently maintained primarily as Word or PDF documents, although their underlying information is largely structured. This thesis investigates how a web-based system for managing module handbooks can be designed and implemented based on user requirements. Requirements were elicited iteratively through stakeholder discussions and refined with clickable prototypes. Based on these findings, a minimum viable product (MVP) was developed that supports structured module and modulebook management, change tracking and saved modulebook versions, import of existing module handbook PDFs, and generation and export of printable module handbooks. The system is implemented with Spring Boot, PostgreSQL, Spring JDBC, and Flyway, complemented by a Python/FastAPI service for PDF extraction and deployed in the BHT Kubernetes environment. OpenTelemetry provides basic technical observability. A requirements-based evaluation shows that the core functional requirements were implemented, while usability was only partially evaluated and several workflow features remain outside the MVP scope. The work demonstrates how structured data can support more maintainable and traceable module handbook management.

Contents

1. Introduction **7**

1.1. Background and Problem Statement 7
1.2. Motivation 8
1.3. Research Question 9
1.4. Goal and Scope 9
1.5. Methodology 9

2. User Requirements **10**

2.1. Stakeholders and User Roles 10
2.2. Requirements Elicitation Process 11
2.2.1. Stage 1: Understanding the Existing Process and Basic Requirements 11
2.2.2. Stage 2: Prototype-Based Requirements Refinement 12
2.2.3. Stage 3: MVP Definition 13
2.3. Functional Requirements 14
2.3.1. FR1: Module Handbook Management 14
2.3.2. FR2: Version Control 14
2.3.3. FR3: Module Handbook Import 15
2.3.4. FR4: Module Handbook Generation and Export 15
2.4. Non-functional Requirements 16

3. Product Development **17**

3.1. Product Concept and Interaction Flow 17
3.2. FR1 – Module Handbook Management 17
3.2.1. Modulebook Workspace 17
3.2.2. Module Overview and Management 18
3.2.3. Structured Module Representation 19
3.2.4. Supportive Module Editing 20
3.2.5. Module Status 21
3.3. FR2 – Version Control 22
3.3.1. Versioning Model 22
3.3.2. Version Comparison 23
3.3.3. Restore 24

3.4. FR3 – Module Handbook Import . . . 25
3.4.1. Import Workflow . . . 25
3.4.2. Field Mapping and Re-import . . . 26
3.5. FR4 – Module Handbook Generation and Export . . . 26
3.5.1. Generate Module Handbook View . . . 26
3.5.2. Modulebook Export . . . 27

4. Technical Implementation **28**
4.1. System Architecture . . . 28
4.2. Technology Stack . . . 29
4.3. Data Model and Database Implementation . . . 29
4.3.1. Conceptual ER Model . . . 29
4.3.2. Database ER Diagram . . . 30
4.3.3. Module Data Model . . . 32
4.3.4. Version-Control Data Model . . . 32
4.4. FR1 – Module Handbook Management Implementation . . . 33
4.5. FR2 – Version Control Implementation . . . 33
4.5.1. Change-Set Implementation . . . 33
4.5.2. Saved Modulebook Version Implementation . . . 34
4.5.3. Supported Version-Control Operations . . . 34
4.6. FR3 – Module Handbook Import Implementation . . . 36
4.7. FR4 – Module Handbook Generation and Export Implementation . . . 40
4.8. Validation, Error Handling, and Testing . . . 40
4.9. CI/CD, Deployment, and Observability . . . 41

5. Evaluation **43**
5.1. Evaluation against MVP Functional Requirements . . . 43
5.2. Evaluation against Non-functional Requirements . . . 44
5.3. Requirements Outside the MVP Scope . . . 44

6. Conclusion **46**

References **47**

A. Supplementary Material **48**
A.1. Links . . . 48
A.2. Initial Requirements Meeting Protocol . . . 48
A.3. User Meeting: Figma Test Protocol . . . 49

List of Figures

1.1. Example of an existing module handbook and an individual module description in BHT. 7

2.1. Screenshots of the first clickable Figma prototype used during the initial requirements elicitation stage. 11
2.2. Figma design of the workload field 13
2.3. Figma design of the version-control concept 13

3.1. Modulebook workspace with module management, import, version-control, and export functions. 18
3.2. Module overview and management interface. 19
3.3. Module editing interface with structured workload input, calculated values, and contextual support. 21
3.4. Change history containing saved modulebook versions and intermediate change sets. 23
3.5. Comparison of changes between two version points. 24
3.6. Restore action for reverting the modulebook to an earlier state. 24
3.7. Module handbook PDF import interface (left: report; right: user review and edit). 25
3.8. Generated module handbook view data. 26
3.9. Print-ready view of the current modulebook. 27

4.1. System architecture of the module handbook management system. . . . 28
4.2. Conceptual ER model of the module handbook system. 30
4.3. Relational database ER diagram of the implemented system. 31
4.4. Simplification of module retrieval through the `module_view` database view. 32
4.5. PDF import pipeline from the source document to structured module data. 36
4.6. Import result: the module number `B23` was failed to be detected 38
4.7. Import issue caused by an irregular table structure: the module number `B23` was merged into title content, so the pipeline failed to detect the start of a new module. 38
4.8. GitLab CI/CD pipeline overview. 42

List of Tables

2.1. Stakeholders and their responsibilities in the module handbook process. 10

3.1. Field in the current system and corresponding field in the existing module handbook PDFs 20

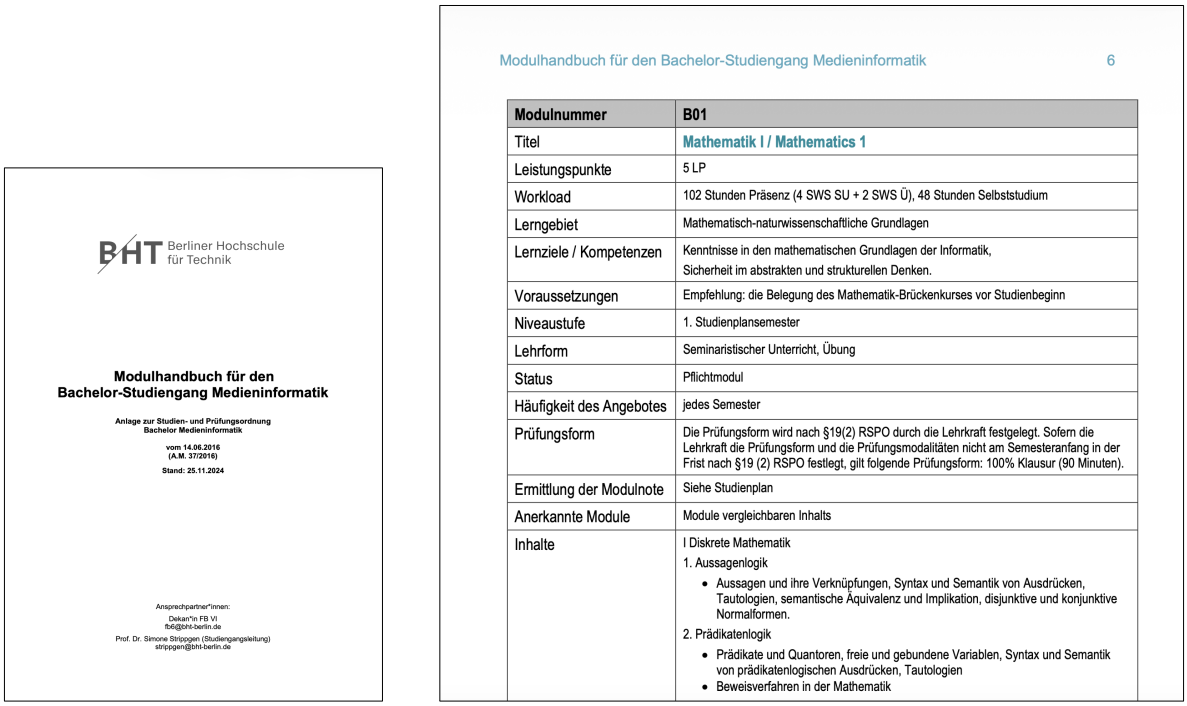
4.1. Technology stack of the implemented system. 29
4.2. Supported version-control operations. 34
4.3. Comparison between the actual number of modules and the number of modules detected by the import pipeline. 38
4.4. Examples of frontend and backend validation rules. 41

5.1. Evaluation against MVP functional requirements. 43
5.2. Evaluation against non-functional requirements. 44
5.3. Requirements outside the MVP scope. 45

1. Introduction

1.1. Background and Problem Statement

Module handbooks. A module handbook describes the modules of a study program. It contains information such as module titles, credits, learning objectives, contents, workload, and so on. It serves as an information source for students, teaching staff, program coordinators, and administrative bodies. Figure 1.1 shows an example of an existing module handbook and an individual module description.



BHT Berliner Hochschule für Technik

Modulhandbuch für den Bachelor-Studiengang Medieninformatik

Anlage zur Studien- und Prüfungsordnung Bachelor Medieninformatik

vom 14.06.2016 (A.M. 37/2016)

Stand: 25.11.2024

Ansprechpartner*innen:
Dekan*in FB VI
fb6@bht-berlin.de
Prof. Dr. Simone Strippgen (Studiengangsleitung)
strippgen@bht-berlin.de

Modulhandbuch für den Bachelor-Studiengang Medieninformatik 6

Modulnummer	B01
Titel	Mathematik I / Mathematics 1
Leistungspunkte	5 LP
Workload	102 Stunden Präsenz (4 SWS SU + 2 SWS Ü), 48 Stunden Selbststudium
Lerngebiet	Mathematisch-naturwissenschaftliche Grundlagen
Lernziele / Kompetenzen	Kenntnisse in den mathematischen Grundlagen der Informatik, Sicherheit im abstrakten und strukturellen Denken.
Voraussetzungen	Empfehlung: die Belegung des Mathematik-Brückenkurses vor Studienbeginn
Niveaustufe	1. Studienplansemester
Lehrform	Seminaristischer Unterricht, Übung
Status	Pflichtmodul
Häufigkeit des Angebotes	jedes Semester
Prüfungsform	Die Prüfungsform wird nach §19(2) RSPO durch die Lehrkraft festgelegt. Sofern die Lehrkraft die Prüfungsform und die Prüfungsmodalitäten nicht am Semesteranfang in der Frist nach §19 (2) RSPO festlegt, gilt folgende Prüfungsform: 100% Klausur (90 Minuten).
Ermittlung der Modulnote	Siehe Studienplan
Anerkannte Module	Module vergleichbaren Inhalts
Inhalte	I Diskrete Mathematik 1. Aussagenlogik • Aussagen und ihre Verknüpfungen, Syntax und Semantik von Ausdrücken, Tautologien, semantische Äquivalenz und Implikation, disjunktive und konjunktive Normalformen. 2. Prädikatenlogik • Prädikate und Quantoren, freie und gebundene Variablen, Syntax und Semantik von prädikatenlogischen Ausdrücken, Tautologien • Beweisverfahren in der Mathematik

Figure 1.1.: Example of an existing module handbook and an individual module description in BHT.

Current document-based workflow. At Berliner Hochschule für Technik (BHT), module handbooks are currently maintained primarily as Word or PDF documents rather than as structured application data. A professor or module coordinator submits new or modified module information, often in the form of a table by email. The Studiengangsleitung reviews the content and integrates the changes into the existing

module handbook. Referat Studium und Lehre performs formal and regulatory review, and further academic approval and publication steps may follow depending on the process.

Consequently, changing module information involves more than modifying the content itself. Information must also be transferred between documents, formatted, checked for consistency, incorporated into a new document version, and reviewed again.

Manual and error-prone maintenance. The repeated transfer of information between people and documents can lead to copy-and-paste errors, omitted information, inconsistent formatting, and additional manual checking. Furthermore, even a small change to a single field of one module results in a new version of the module handbook.

The maintenance process can also involve several review and approval steps before an updated module handbook is published, which increases the effort required for changes.

Inconsistent structure and terminology. The existing module handbooks are not completely uniform across study programs. The same semantic concept may be represented by different labels. For example, “Module ID”, “Modulkürzel”, and “Modulnummer” may all refer to the identifier of a module.

The set of fields can also differ between module handbooks. For example, some module handbooks contain room requirements or a field such as “Verwendbarkeit”, while others do not. In addition, the same module information may appear in several parts of a module handbook, such as the *Modulverzeichnis*, the *Studienplan*, and the individual module descriptions. Such redundant information has to remain synchronized manually.

These characteristics make module handbook maintenance difficult to manage consistently and make individual changes difficult to trace across document versions.

1.2. Motivation

A key observation underlying this thesis is that module handbooks are published as documents, while much of their underlying information is structured. This observation emerged from reviewing multiple NC Bachelor module handbooks and from discussions with professors and representatives of Referat Studium und Lehre.

Despite differences in terminology and individual fields, the reviewed module handbooks showed a recurring structure: a module handbook consists of modules, and modules are described through recurring fields.

This motivates a separation between data and presentation. Module information can be maintained as structured application data, while the PDF becomes a generated presentation and publication format. Such a separation can improve consistency,

maintainability, traceability, and document generation. At the same time, the resulting system should reflect the actual requirements and workflows of its users.

1.3. Research Question

Based on the identified problems and motivation, this thesis addresses the following research question:

> *How can a web-based system for managing module handbooks be designed and implemented based on the requirements of its users?*

1.4. Goal and Scope

The goal of this thesis is to design and implement a minimum viable product (MVP) for digitizing module handbook management. The MVP focuses on the following core functions:

- structured representation of module data,
- module handbook management,
- version control,
- import of existing module handbook PDFs, and
- generation and export of module handbooks.

1.5. Methodology

This thesis follows an iterative, requirements-driven development process. First, stakeholder discussions were conducted to understand the existing module handbook workflow and identify user requirements. Based on these findings, clickable prototypes were developed and refined iteratively with stakeholder feedback.

The resulting requirements were then translated into a product concept and subsequently into a technical design and MVP implementation. The implemented system is finally evaluated against the functional and non-functional requirements defined for the MVP.

2. User Requirements

2.1. Stakeholders and User Roles

Table 2.1 summarizes the main stakeholders identified during the requirements elicitation process and their responsibilities in the module handbook workflow.

Table 2.1.: Stakeholders and their responsibilities in the module handbook process.

Role / Stakeholder	**Meaning in the system**	**Main responsibility**
Professor / Module Coordinator	Person responsible for a module	Creates and edits module descriptions and maintains the academic content of the module.
Program Director (*Studiengangsleitung*)	Person responsible for the academic quality of a study program	Reviews module content for academic coherence, completeness, and suitability for the respective study level.
Referat Studienangelegenheiten und Lehre (RefSL)	Administrative and regulatory review body	Reviews module descriptions for formal and regulatory compliance, for example whether requirements for Bachelor or Master modules are fulfilled.
Kommission Studium und Lehre (Kommission SL)	Academic review body	Reviews proposed changes and provides recommendations for approval.
Senate (*Senat*)	Higher-level approval body	Provides formal approval or confirmation where required.
Student	Read-only user / viewer	Views and downloads published module handbooks.
Administrator	Technical system administrator	Manages users, roles, templates, and system settings.

2.2. Requirements Elicitation Process

The requirements were identified and refined iteratively through stakeholder discussions and prototype-based feedback. The process was organized into three stages: understanding the existing workflow, refining requirements through a clickable prototype, and reducing the resulting requirements to a technically feasible minimum viable product (MVP).

2.2.1. Stage 1: Understanding the Existing Process and Basic Requirements

Discussions were conducted with core stakeholders, including a professor and representatives of Referat Studium und Lehre. The corresponding meeting protocol is included in Appendix A.2.

The initial requirements included:

- structured representation of module data,
- user and permission concepts, and
- collaborative editing.

Based on these findings, a first clickable Figma prototype was created. The prototype focused primarily on representing module information as explicit structured fields. (Figma clickable prototype. Figure 2.1)

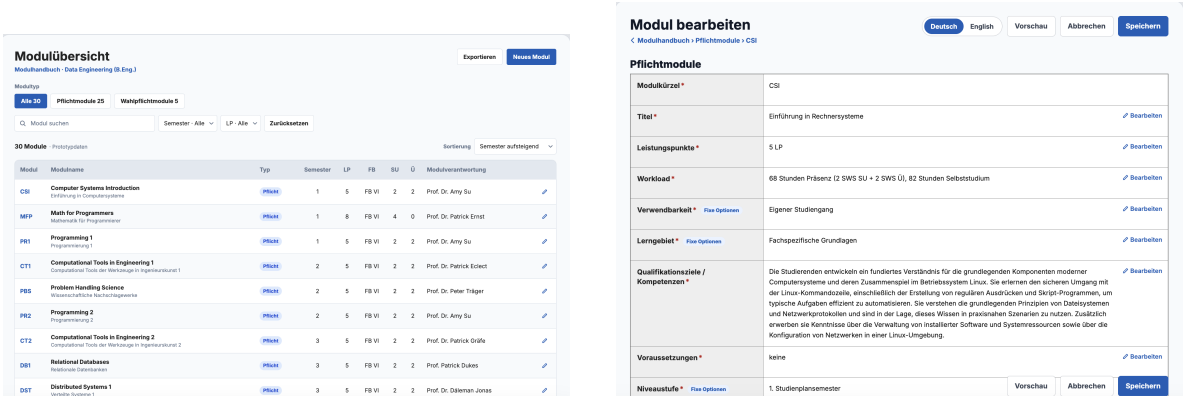


Figure 2.1.: Screenshots of the first clickable Figma prototype used during the initial requirements elicitation stage.

2.2.2. Stage 2: Prototype-Based Requirements Refinement

In the second stage, the prototype was reviewed together with stakeholders to refine the requirements (interview protocol see Appendix A.3). The discussions showed that several module fields could not be represented adequately as simple free-text inputs.

The workload field was one example. It combines module credits, teaching forms, SWS values, presence hours, and self-study hours. These values are interdependent rather than independent input fields.

At BHT, one Leistungspunkt (LP) currently corresponds to 30 hours of workload. The total workload of a module is therefore derived from its credits:

$$\text{Total Workload} = \text{Credits} \times 30 \text{ hours}$$

For the calculation of presence hours, the SWS value (*Semesterwochenstunden*) is the leading input. One SWS corresponds to 45 minutes per week, and the calculation is based on 17 teaching weeks per semester. The presence hours are therefore derived from the SWS values of the selected teaching forms:

$$\text{Presence Hours} = \sum_{i=1}^{n} (\text{SWS}_i \times 0.75 \text{ hours} \times 17)$$

where each i represents one selected teaching form.

The self-study hours are then calculated as the remaining part of the total workload:

$$\text{Self-Study Hours} = \text{Total Workload} - \text{Presence Hours}$$

This example illustrates that a module field may require more than a single stored value. It may also involve predefined options, validation rules, contextual help, calculated values, and dependencies between multiple fields.

Another important aspect is the need for version control. Stakeholders required changes to remain traceable and relevant module handbook states to be saved for later publication.

Based on this feedback, the prototype was refined and the core system functions were further developed in Figma (Core function design in Figma). The workload field was redesigned to represent its underlying structure and dependencies more explicitly (Figure 2.2).

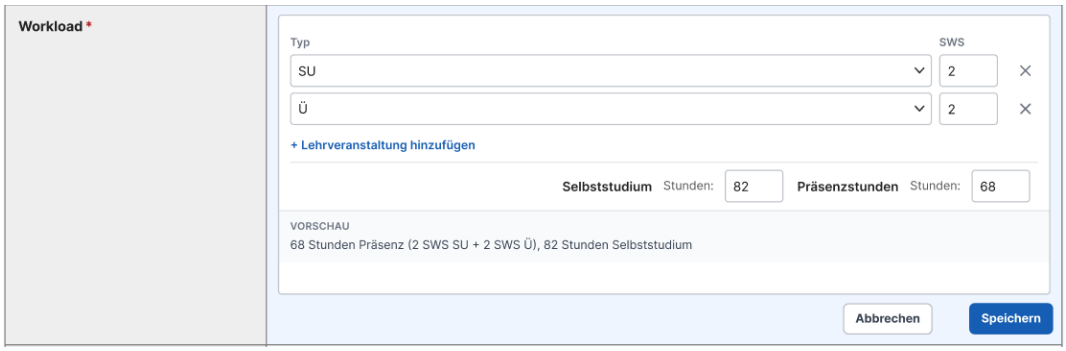


Figure 2.2.: Figma design of the workload field

Inspired by Git, a version-control concept was designed to make module changes traceable and to support the comparison and restoration of previous states.(Figure 2.3).

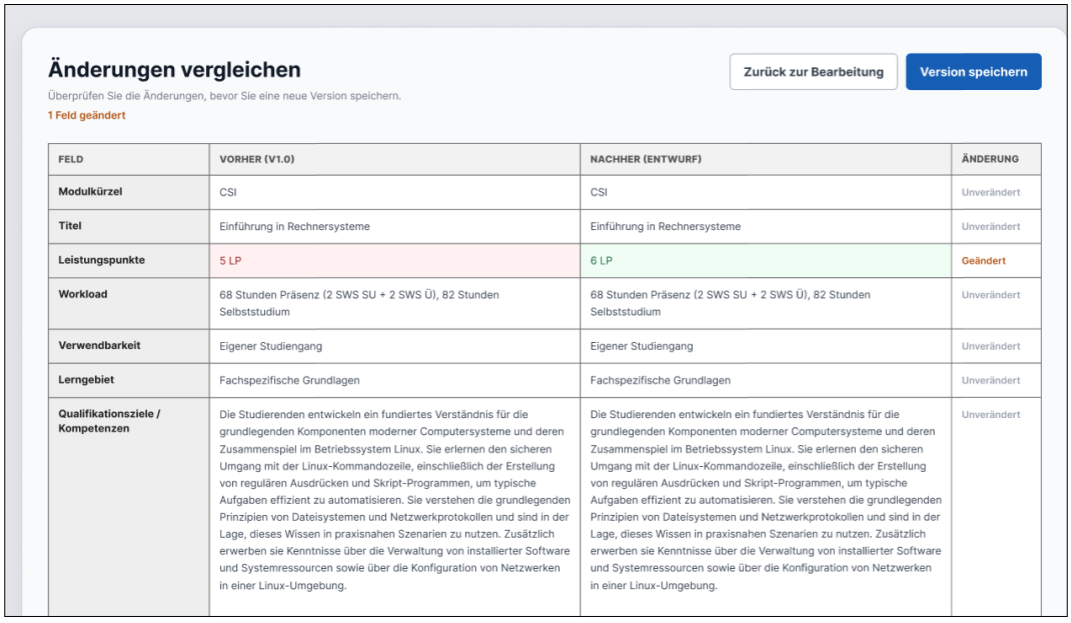


Figure 2.3.: Figma design of the version-control concept

2.2.3. Stage 3: MVP Definition

The third stage focused on reducing complexity and defining a technically feasible MVP. Stakeholders reviewed the proposed UI design, and the following functional areas were selected as the core scope of the implementation:

- module handbook and module management,
- version control,
- import of existing module handbook PDFs, and

- generation and export of module handbooks.

Additional requirements, including complete authentication, role-based permissions, collaborative editing, and a full approval workflow, were identified as relevant but deferred from the MVP.

2.3. Functional Requirements

The four functional requirement groups are further divided into the following sub-requirements.

2.3.1. FR1: Module Handbook Management

FR1.1 Structured Module Data.
Module information shall be stored as structured data.

FR1.2 Module Handbook Management.
Users shall be able to view, create, and delete module handbooks.

FR1.3 Module Management.
Users shall be able to view, create, update, and delete modules belonging to a selected module handbook.

FR1.4 Input Assistance and Validation.
The system shall support users with predefined values, validation rules, warnings, and contextual help where required.

FR1.5 Module Status Management.
Users shall be able to change the lifecycle status of a module, for example between active and deactivated states.

2.3.2. FR2: Version Control

FR2.1 Change Traceability.
Changes to modules and module handbooks shall be traceable.

FR2.2 Change Sets.
One user operation may contain one or more field-level changes and shall be represented as one logical change set.

FR2.3 Save Module Handbook Version.

Users shall be able to save the current state of a module handbook as a named version.

FR2.4 Version History.

Users shall be able to view previous changes to modules and module handbooks.

FR2.5 Version Comparison.

Users shall be able to compare different states or versions.

FR2.6 Restore Previous State.

Users shall be able to restore a previous state where supported by the versioning model.

2.3.3. FR3: Module Handbook Import

FR3.1 PDF Import.

Users shall be able to import an existing module handbook PDF.

FR3.2 Module and Field Detection.

The system shall extract modules and map detected content to structured module fields.

FR3.3 Initial Imported Version.

The first successful import shall establish an initial version of the module handbook.

FR3.4 Re-import and Update.

Existing module handbooks shall be updatable through later imports where corresponding modules can be identified.

2.3.4. FR4: Module Handbook Generation and Export

FR4.1 Generate Module Handbook View.

The system shall generate a consistent module handbook representation from structured data.

FR4.2 Export Current Module Handbook.

Users shall be able to print or export the current module handbook.

FR4.3 Export Saved Module Handbook Version.

Users shall be able to print or export a previously saved module handbook version.

2.4. Non-functional Requirements

The following non-functional requirements complement the functional requirements and describe quality-related goals of the MVP.

NFR1: Usability.
The system should remain understandable and practical for occasional users. In particular, the interface should support users through structured input, contextual information, warnings, and familiar representations of module handbook content.

NFR2: Maintainability.
The implementation should be modular and extensible so that additional fields and functions can be added later.

NFR3: Deployability and Portability.
The system should be deployable in the BHT Kubernetes environment.

NFR4: Observability and Technical Traceability.
The system should provide basic technical traceability of backend requests and service interactions to support monitoring and debugging.

3. Product Development

3.1. Product Concept and Interaction Flow

The product concept translates the requirements identified in Chapter 2 into a concrete interaction flow:

Create Modulebook
↓
(Import Existing PDF and Create Initial Version)
↓
Read / Create / Update / Delete Modules
↓
Save Changes
↓
View Change History / Compare / Restore
↓
Save a Modulebook main Version
↓
View / Export Module Handbook

The following sections discuss the product development in relation to the corresponding user requirements.

3.2. FR1 – Module Handbook Management

3.2.1. Modulebook Workspace

Each modulebook represents the workspace of one study program. The workspace groups the modules of the program together with the corresponding import, version-control, and export functions. Core modulebook operations are provided according to FR1.2.

An additional Study Program layer was intentionally omitted to keep the main interaction centered on the modulebook (Figure 3.1).

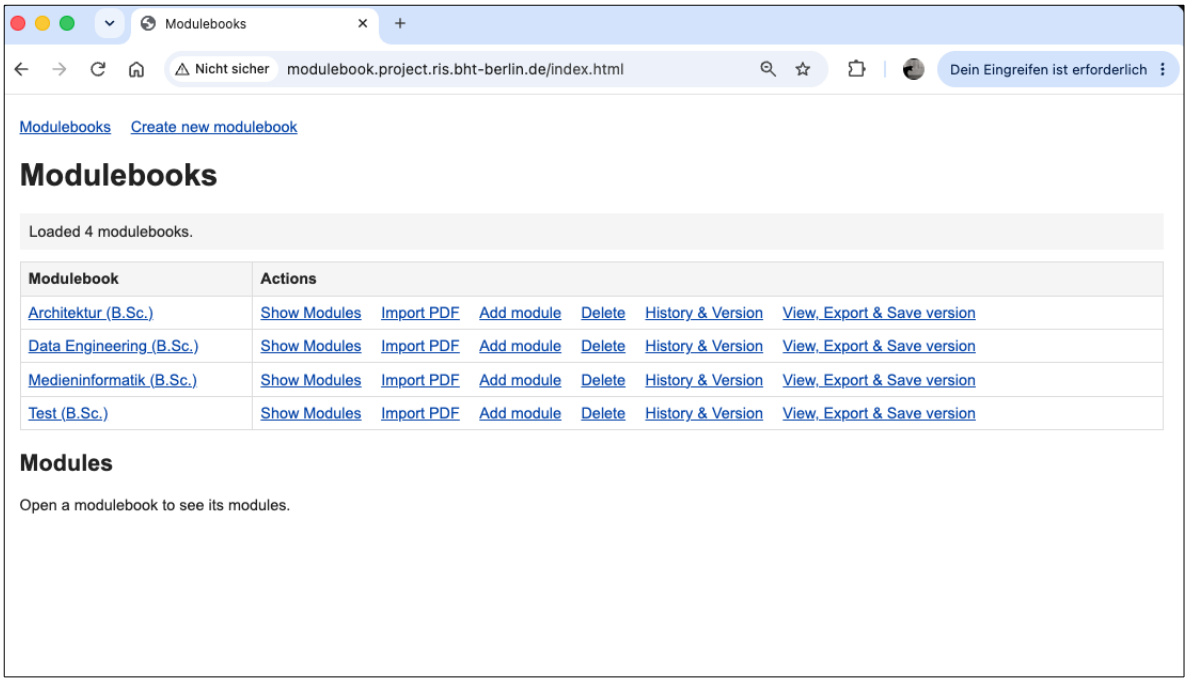


Figure 3.1.: Modulebook workspace with module management, import, version-control, and export functions.

3.2.2. Module Overview and Management

After selecting a modulebook, the system displays all modules belonging to it (Figure 3.2). The overview presents key information such as module number, title, credits, status, and available actions. Users can open a module, edit it, inspect its history, change its status, or delete it according to FR1.3.

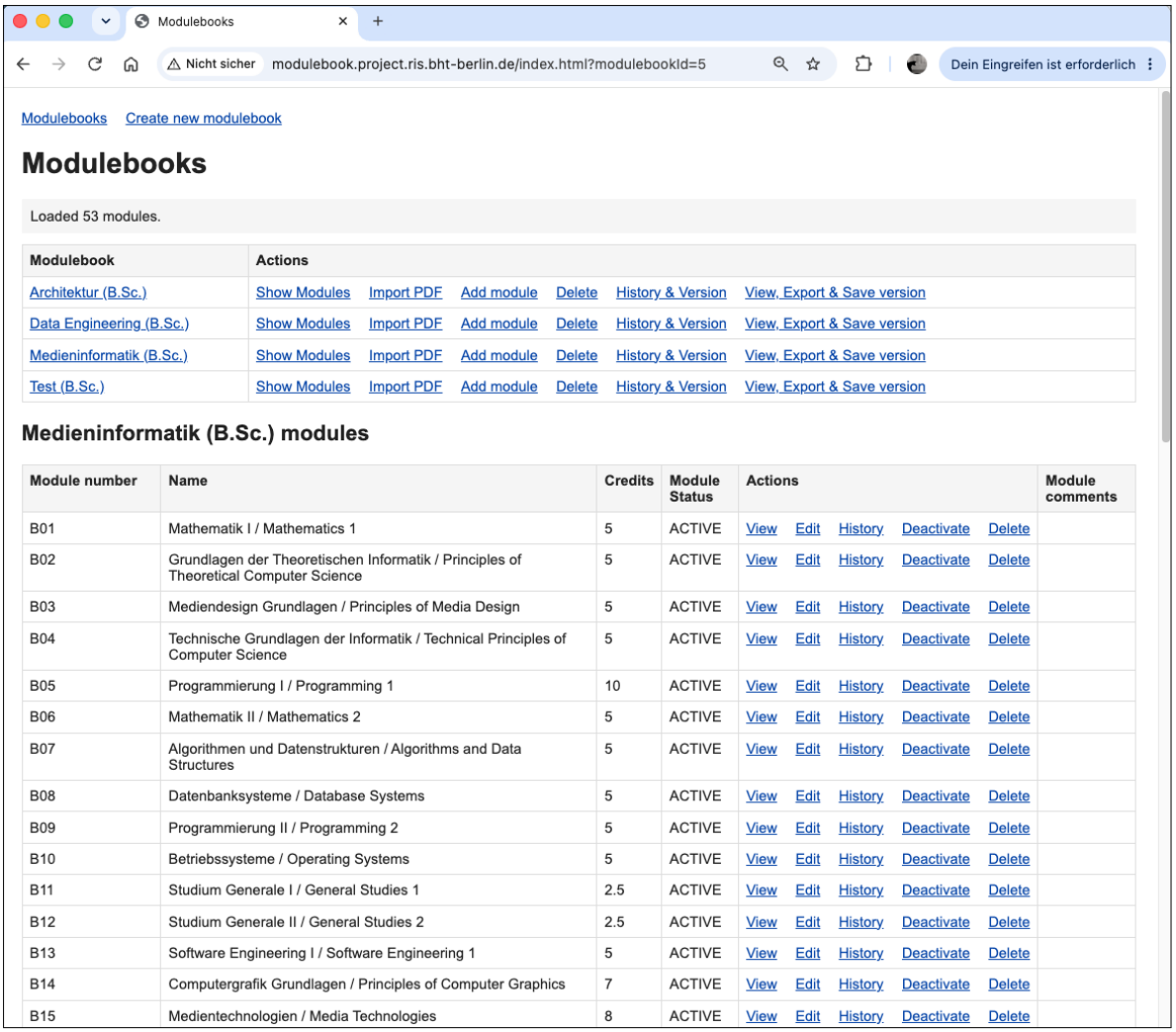


Figure 3.2.: Module overview and management interface.

3.2.3. Structured Module Representation

Module information is represented through explicit fields. The field structure is based on the recurring structure identified in the reviewed module handbooks and refined through stakeholder discussions (FR1.1).

Compared with the original module handbook, the digital module representation adds two new fields, “Coordinator” and “Module weight”. The remaining fields are derived from the existing module handbook (Table 3.1).

Table 3.1.: Field in the current system and corresponding field in the existing module handbook PDFs

No.	Field in the system	Field in existing modulebook
1	Module number	Modulnummer; Module ID; Modul ID
2	Title	Titel; Title
3	Credits	Leistungspunkte; Credits; Credit Points
4	Workload	Workload; Präsenzzeit; Präsenszeit
5	Applicability	Verwendbarkeit
6	Learning area	Lerngebiet; Learning Area;
7	Learning objectives	Lernziele / Kompetenzen; Qualifikationsziele / Kompetenzen; Learning objectives and competencies
8	Prerequisites	Voraussetzungen; Pre-conditions
9	Level	Niveaustufe; Level
10	Teaching and learning form	Lehrform; Lernform; Lehr- und Lernform; Teaching and learning forms
11	Module type	Status
12	Frequency	Häufigkeit des Angebotes; Turnus; Frequency of module offer
13	Examination form	Prüfungsform
14	Contents	Inhalte
15	Literature	Literatur
16	Additional notes	Weitere Hinweise
17	Coordinator	Koordinator/in (usually on the first few pages of the modulebook)
18	Module weight	Module Gewischt ((in the annex to the study and examination regulations)

3.2.4. Supportive Module Editing

The module editing interface follows the familiar table-based representation of existing module handbooks while adding structured input and assistance (NFR1, FR1.4).

Not all fields are treated as free-text inputs. Depending on the field, the interface provides required-field indicators, predefined options, contextual help, warnings, and calculated values.

For example, for the workload field, at first users select one or more teaching forms and enter the corresponding SWS values. The interface derives the presence hours and calculates an indication of the corresponding self-study hours based on the module credits.

Hints and warnings are displayed when the entered values do not correspond to the expected workload. The teaching-form is connected to the input of the workload and is automatically selected. Contextual information further supports users in understanding the available choices (Figure 3.3).

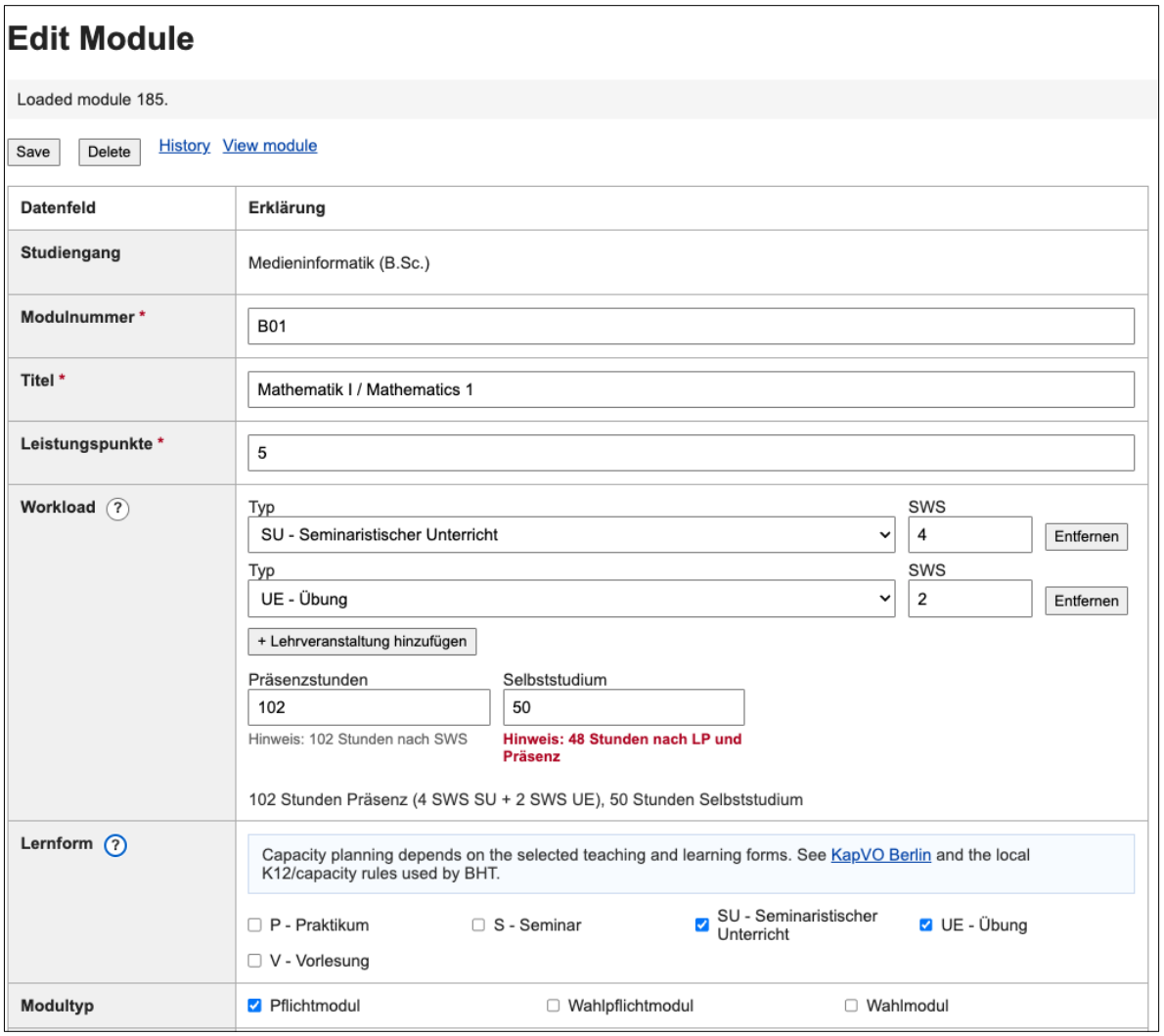


Figure 3.3.: Module editing interface with structured workload input, calculated values, and contextual support.

3.2.5. Module Status

Modules can have different lifecycle states. In the current implementation, a module can be active or deactivated. Active modules are included in the current module handbook output, while deactivated modules remain stored but are excluded from normal publication output.

The status can be changed from the module overview. This allows users to control whether a module belongs to the current publication state without removing its data

from the system (FR1.5).

3.3. FR2 – Version Control

3.3.1. Versioning Model

The versioning concept distinguishes between two types of version points:

- automatically recorded change sets, and
- user-saved main modulebook versions.

A change set represents one user-visible operation and can contain one or multiple field-level changes within a module. For example credits, workload, or teaching forms. A change set can also represent operations such as creating a module, changing a module status, importing a PDF, or restoring a previous state (FR2.1, FR2.2).

A saved modulebook version represents a deliberately stored state of the complete modulebook, such as the initial imported version or a later publication version corresponding to an *Amtliche Mitteilung* (FR2.3). A saved version can contain a version message and publication-related metadata, such as the modulebook release date and the date of the corresponding study and examination regulations. Saved versions can be viewed independently from the current state, opened for printing or export, and used as restore targets.

Separating automatically recorded change sets from user-saved modulebook versions supports both detailed traceability of individual changes and reliable preservation of complete historical states. The user-facing version numbers follow a chronological sequence. Saved modulebook versions use major version numbers such as 1.0 and 2.0, while intermediate change sets use minor version numbers such as 1.1 and 1.2.

For example:

2.0 Saved Modulebook Version
1.2 Change Set
1.1 Change Set
1.0 Saved Modulebook Version

The history view presents both types of version points together and shows information such as the creation time, source of the operation, reason for the change, affected modules, and number of individual changes (Figure 3.4).

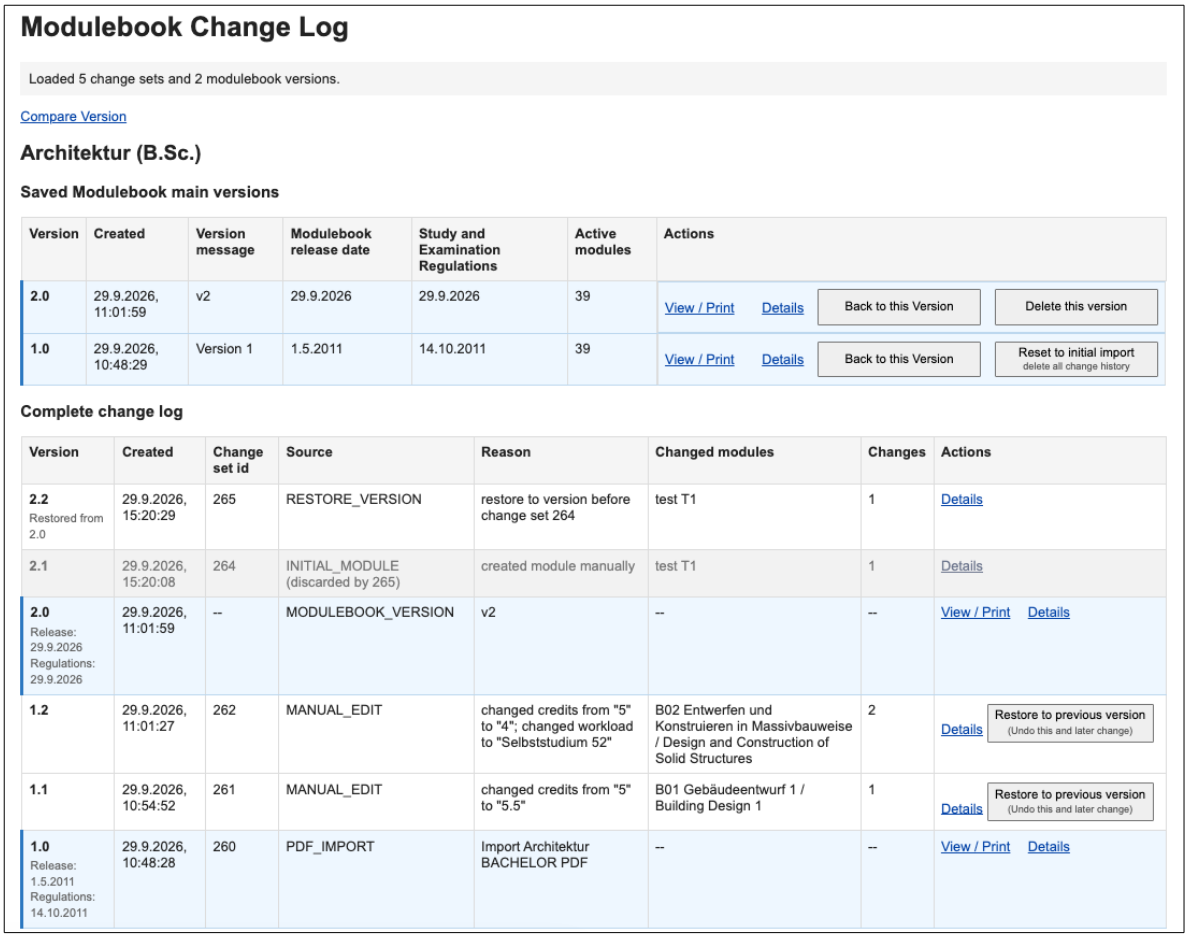


Figure 3.4.: Change history containing saved modulebook versions and intermediate change sets.

3.3.2. Version Comparison

The system allows users to compare different version points (FR2.5). The comparison focuses on the modules and fields that changed between the selected states. This makes individual changes visible without requiring users to manually compare complete PDF documents (NFR1) (Figure 3.5).

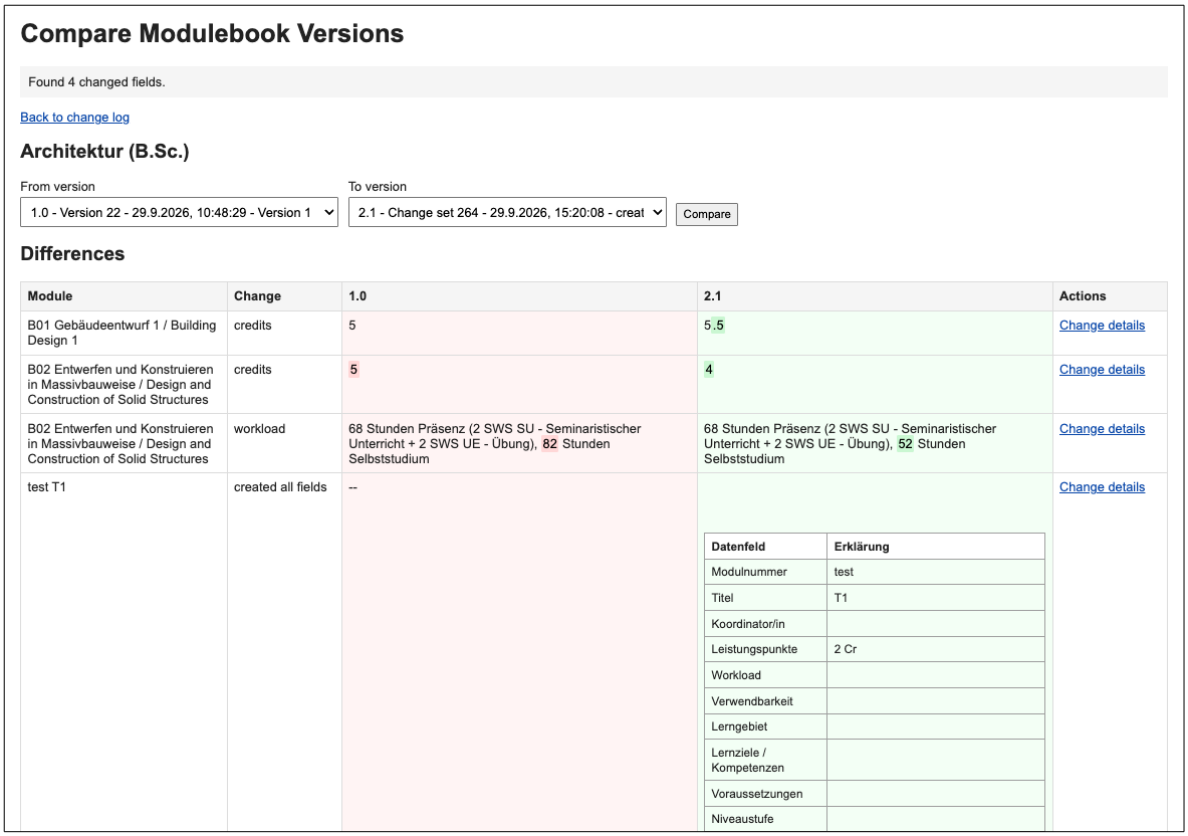


Figure 3.5.: Comparison of changes between two version points.

3.3.3. Restore

Users can restore previous states from the version history (FR2.6).

For a change set, the restore action returns the modulebook to the state before the selected change. Later changes will also be undone (Figure 3.6). For a saved modulebook version, the complete saved state is used as the restore target.

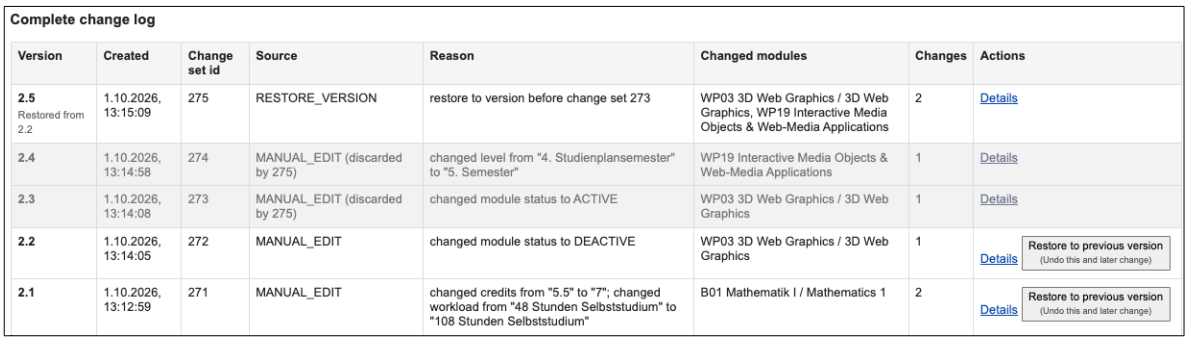


Figure 3.6.: Restore action for reverting the modulebook to an earlier state.

A restore operation itself cannot be undone. Therefore, confirmation steps are provided before a restore operation to warn the user. (NFR1).

3.4. FR3 – Module Handbook Import

3.4.1. Import Workflow

Existing module handbooks can be imported as PDF files at the modulebook level (FR3.1). The purpose of the import function is to migrate existing document-based module information into the structured system without requiring users to enter every module manually.

The general workflow is:

1. Select an existing modulebook.
2. Enter the publication dates and upload a module handbook PDF.
3. Extract and normalize the module data.
4. Review and edit the imported modules.
5. Save the modules to the modulebook.
6. Automatically create the initial imported modulebook version (FR3.3).

The corresponding import interface is shown in Figure 3.7.

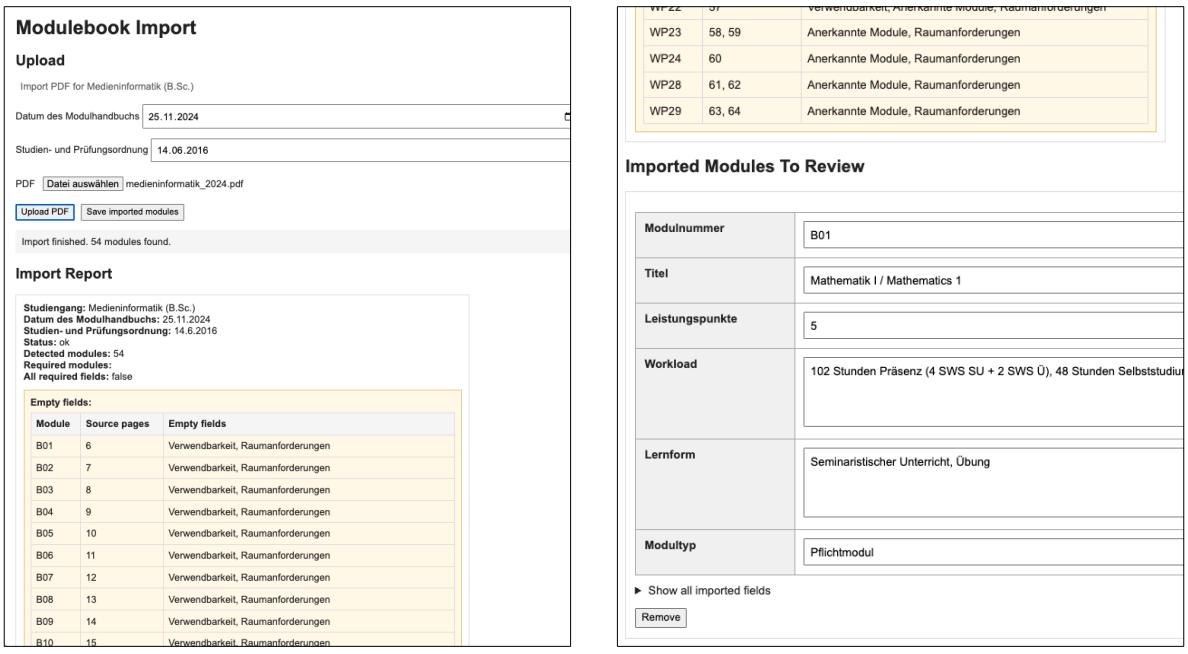


Figure 3.7.: Module handbook PDF import interface (left: report; right: user review and edit).

3.4.2. Field Mapping and Re-import

Different module handbooks may use different labels for the same semantic field. For example, “Module ID”, “Modulkürzel”, and “Modulnummer” can refer to the same module identifier. During import, such labels are mapped to a common structured field (FR3.2).

The same modulebook can also be imported again. Existing modules are matched and updated, newly detected modules are created, and modules that are no longer present in the imported document are marked as “missing from re-import” and deactivated (FR3.4).

The technical extraction and matching process is described in Chapter 4.

3.5. FR4 – Module Handbook Generation and Export

3.5.1. Generate Module Handbook View

The system generates a consistent module handbook representation from the structured module data (FR4.1). Only active modules are included in the current module handbook view (Figure 3.8).

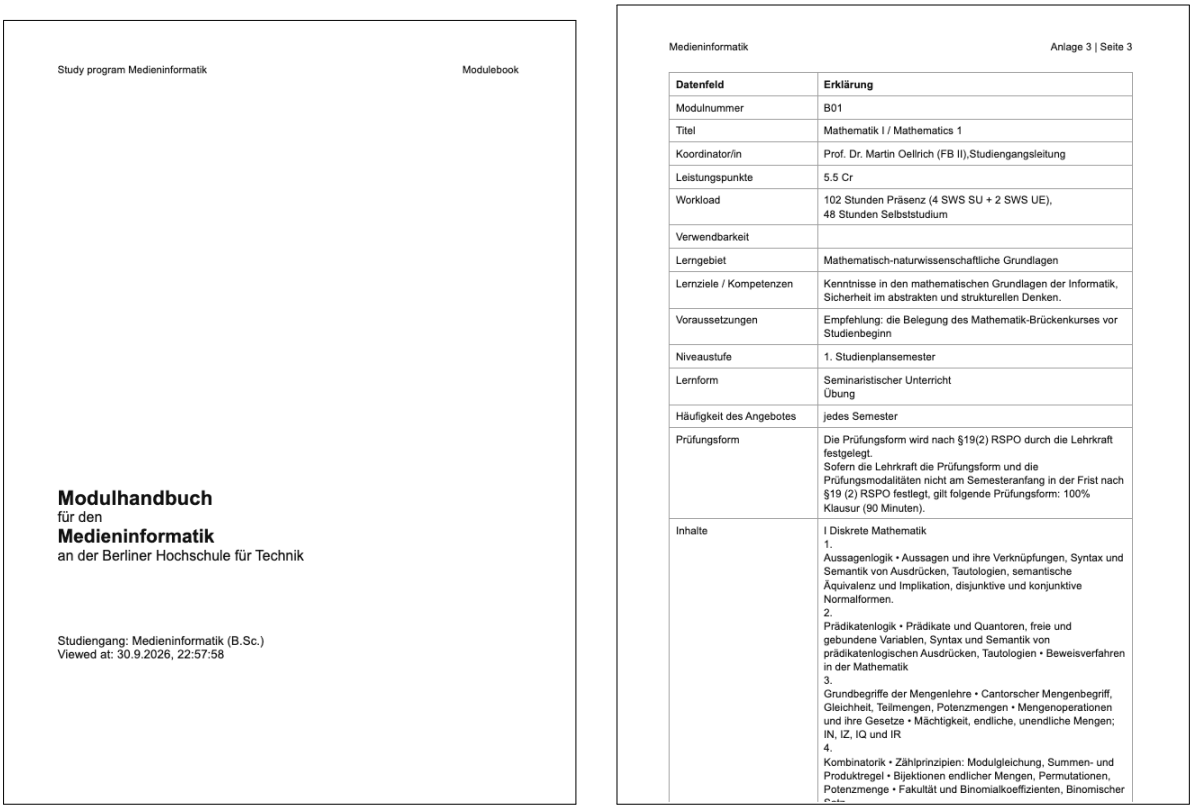


Figure 3.8.: Generated module handbook view data.

3.5.2. Modulebook Export

Users can open the current modulebook state in a print-ready view and print or export it as a PDF (FR4.2).

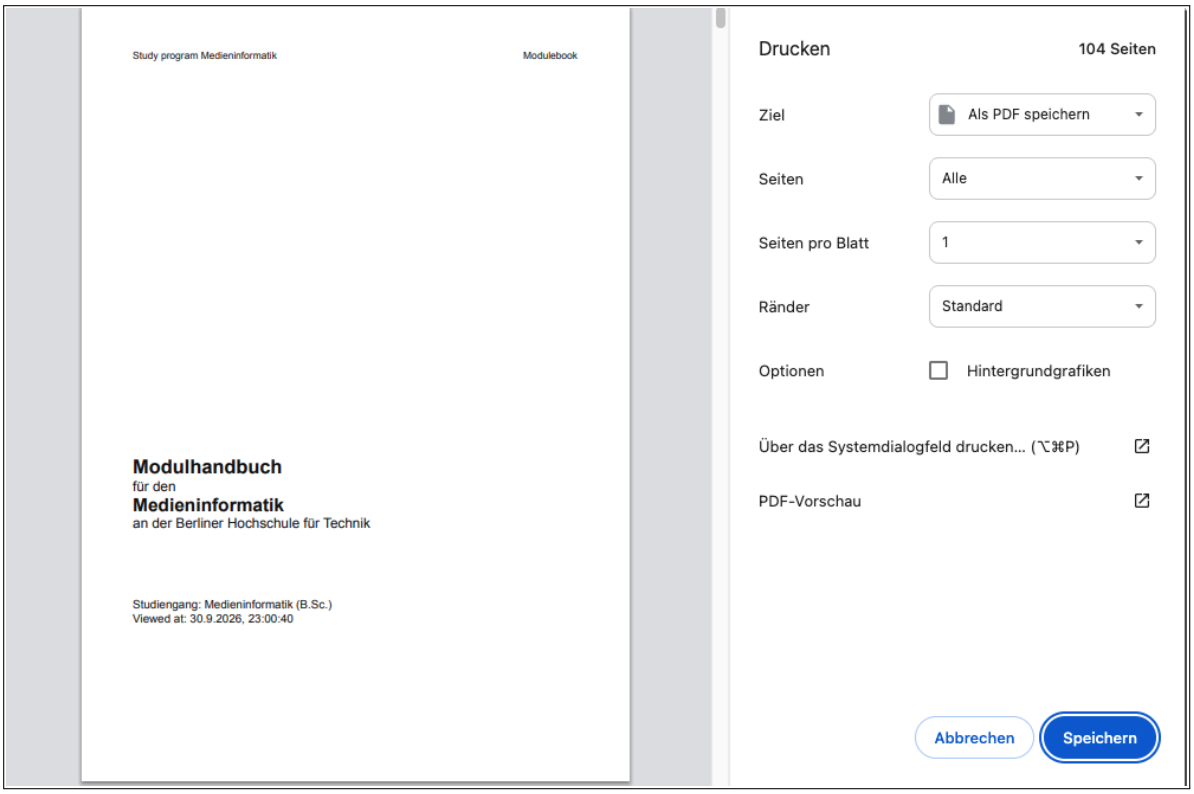


Figure 3.9.: Print-ready view of the current modulebook.

Saved modulebook versions can also be opened independently from the current state and exported (FR4.3). This allows a previously saved publication state to remain reproducible even after later edits have been made to the current modulebook, especially after modules have been deleted.

4. Technical Implementation

4.1. System Architecture

The system follows a client-server architecture (Figure 4.1). Spring Boot serves as the central backend and provides the main application logic and REST interfaces. PostgreSQL stores the structured relational data, including module information and version-control data.

PDF extraction is implemented as a separate Python/FastAPI service because the extraction pipeline relies on Python-based PDF-processing libraries. The separation keeps PDF-specific processing independent from the main Spring Boot application and allows both services to be deployed and maintained separately.

Since the current development stage focuses mainly on the backend, the frontend was implemented using plain HTML, CSS, and JavaScript to keep the client side lightweight and to simplify testing and debugging.

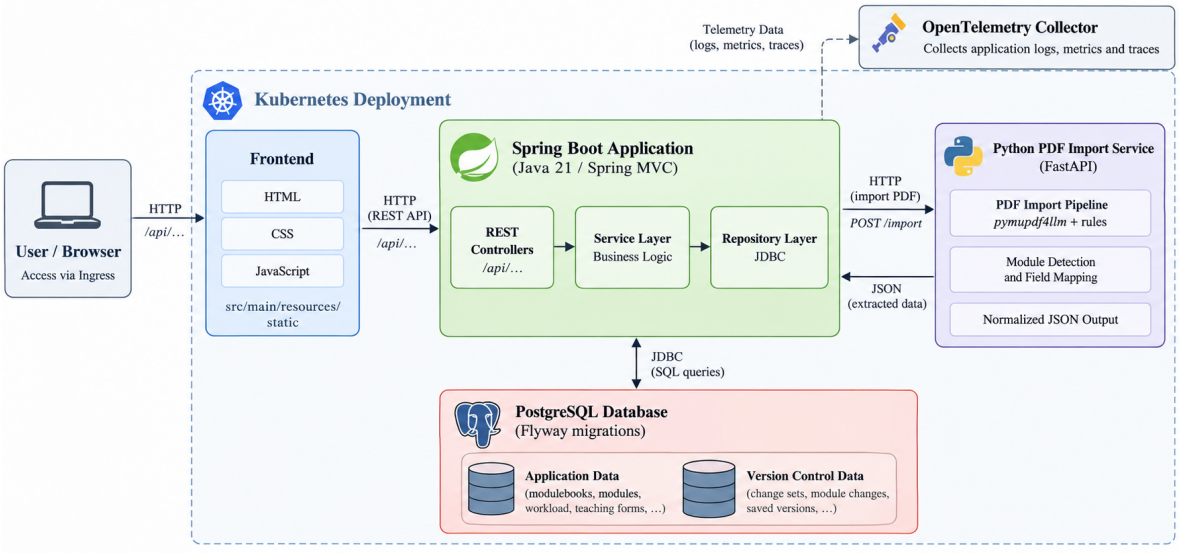


Figure 4.1.: System architecture of the module handbook management system.

4.2. Technology Stack

Table 4.1 summarizes the main technologies used for the implementation and their roles within the system.

Table 4.1.: Technology stack of the implemented system.

Layer / Area	**Technology**	**Purpose**
Backend	Spring Boot, Java 21	Central business logic and REST API.
Database	PostgreSQL	Storage of structured relational module and version-control data.
Database Access	Spring JDBC	Explicit SQL and transparent database operations.
Database Migration	Flyway	Version-controlled and reproducible schema changes supporting NFR2.
PDF Import	Python, FastAPI	Support for Python-based PDF-processing libraries while keeping document processing separate from the main backend.
Frontend	HTML, CSS, JavaScript	Implementation of the MVP user interface without introducing an additional frontend build framework.
Containerization	Docker	Consistent packaging of the application components.
CI/CD	GitLab CI/CD	Automated testing, image building, and deployment supporting NFR3.
Deployment	Kubernetes	Deployment in the BHT Kubernetes environment according to NFR3.
Observability	OpenTelemetry	Application tracing and technical observability according to NFR4.

4.3. Data Model and Database Implementation

4.3.1. Conceptual ER Model

The conceptual ER model represents the main domain entities and their relationships independently of the concrete database implementation. It separates module data, version snapshots, and change tracking as distinct concepts (Figure 4.2).

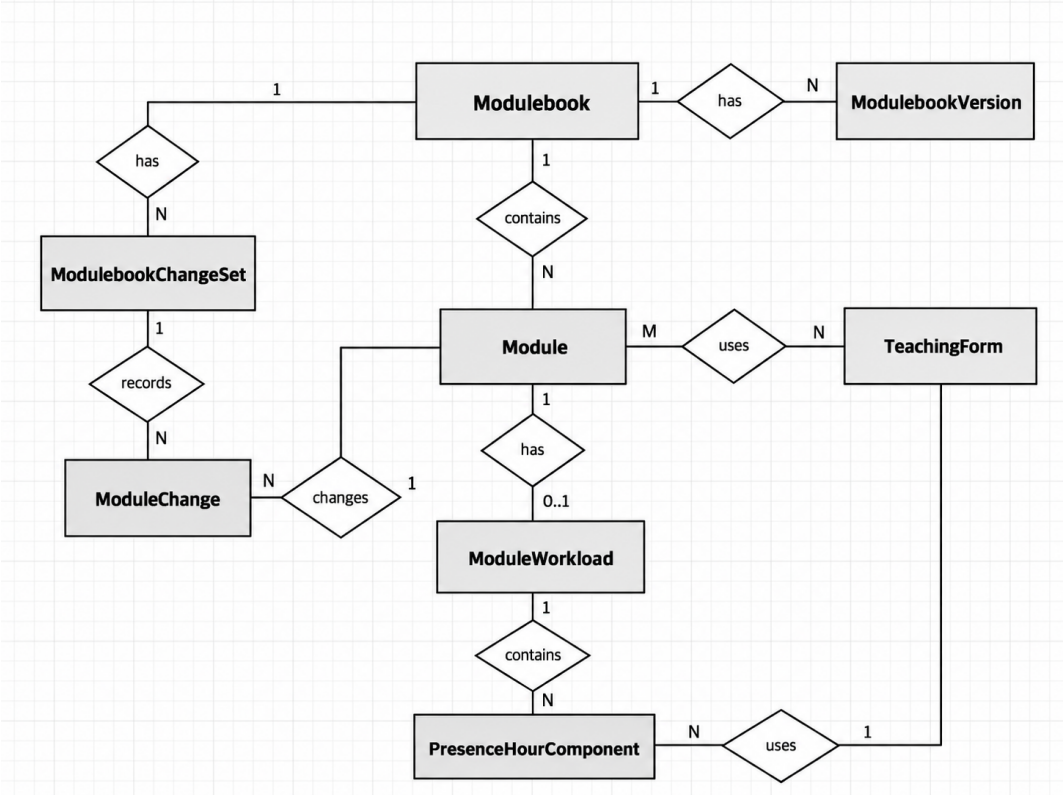


Figure 4.2.: Conceptual ER model of the module handbook system.

4.3.2. Database ER Diagram

The database ER diagram translates the conceptual model into the relational PostgreSQL schema. Additional tables and relationships are introduced to normalize repeated data and support version control (Figure 4.3).

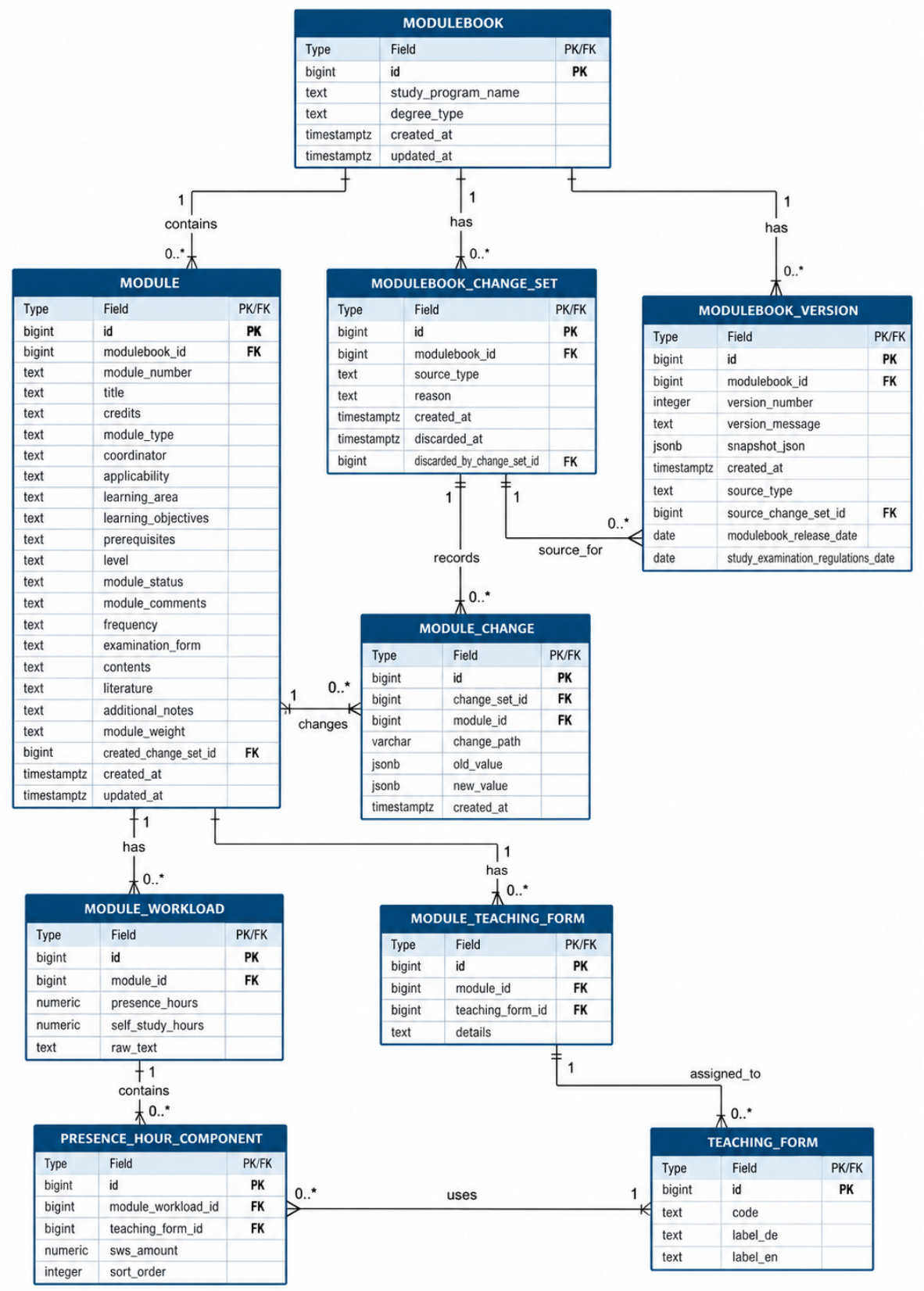


Figure 4.3.: Relational database ER diagram of the implemented system.

4.3.3. Module Data Model

At the current stage, module data is intentionally stored flexibly, particularly because the PDF import process is still evolving and imported module descriptions may contain inconsistent structures. Therefore, strict domain validation is not enforced at the database level. Most descriptive module fields are stored as text so that imported content can be preserved without being rejected too early. The database mainly protects structural consistency through primary keys, foreign-key relationships, and uniqueness constraints.

Most descriptive module fields are stored in the `modules` table, while workload and teaching-form information are modeled separately because they contain structured and repeating data. A module can use multiple teaching forms, and each presence-hour component can reference a teaching form with a specific SWS value.

To simplify read operations, the database view `module_view` combines the normalized module-related tables into one module-oriented representation. The underlying database remains normalized, while the application can retrieve complete module information through a simpler query (Figure 4.4).

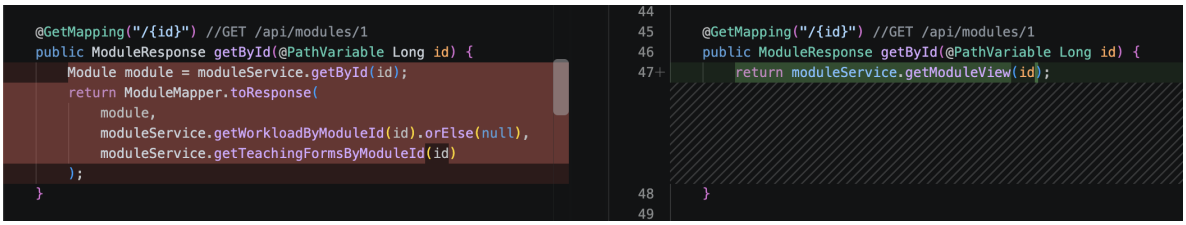


Figure 4.4.: Simplification of module retrieval through the `module_view` database view.

4.3.4. Version-Control Data Model

Change tracking and saved modulebook versions are stored separately in three main tables:

- `modulebook_change_sets` stores one user-visible operation, such as editing a module, importing a PDF, changing a module status, or restoring a previous state;
- `module_changes` stores the concrete field-level changes belonging to a change set, including the old and new values; and
- `modulebook_versions` stores complete snapshots of a modulebook in JSON format at selected points in time.

In principle, the chronological sequence of change sets makes it possible to reconstruct earlier states of a modulebook by applying or reversing recorded field-level changes.

However, relying only on change sets would make recovery fragile in special cases. For example, deleting a module, incomplete change records, or unexpected system errors may leave a past state difficult or impossible to reconstruct reliably. Therefore, the system supports user to save complete versions at specific points in time, for example before publication. These versions are stored as complete JSON snapshots, while change sets provide detailed traceability of the changes made between them.

A straightforward solution was tried at the beginning, in which a complete snapshot of each module would be stored after every change. This would make restoration and comparison simple. However, it would also create substantial data redundancy, since most changes affect only a small part of a module. The implemented design therefore uses change sets for fine-grained chronological tracking and complete snapshots only for selected modulebook versions.

4.4. FR1 – Module Handbook Management Implementation

Module management follows a layered backend structure consisting of Controller, Service, and Repository layers. Controllers handle HTTP requests, Services contain the business logic, and Repositories access PostgreSQL through Spring JDBC.

For example, updating a module may affect several related tables, including `modules`, `module_workloads`, `presence_hour_components`, and `module_teaching_forms`. Before an update, the previous state is loaded so that the new state can be validated and the resulting changes can be recorded in the version-control tables.

4.5. FR2 – Version Control Implementation

4.5.1. Change-Set Implementation

When a user modifies a module, the previous state is loaded before the operation is executed. After the update, the old and new states are compared. One `modulebook_change_sets` record is created for the user operation, while the concrete field-level differences are stored in `module_changes`. Module deletion is handled differently because the module no longer exists in the current module tables after the operation. Before a module is deleted, a complete snapshot of the module is stored as the `old_value`. This preserves the information of the deleted module in the change history.

Restoring a change set reconstructs the modulebook state prior to the selected change. The system uses the stored `old_value` entries in the corresponding module changes to

undo modifications. All later active change sets are also reverted in reverse chronological order, since they depend on the state created after the selected change set.

The restore operation follows several additional rules:

- If a reverted change set contains the creation of a module, the corresponding module is deleted during the restore operation.
- If a reverted change set contains the deletion of a module, the deleted module cannot currently be restored. However, its previous state remains visible in the change history.
- A restore operation itself cannot be undone. Therefore, the user is required to confirm the operation before it is executed (FR2.6).

4.5.2. Saved Modulebook Version Implementation

When a modulebook version is saved, the current state of the complete modulebook is serialized as a JSON snapshot and stored in `modulebook_versions`. The snapshot represents a complete historical state independently from later modifications to the current modulebook.

When a saved modulebook version is opened or restored, the corresponding historical data is loaded from the stored JSON snapshot. This mechanism also enables previously saved publication states to remain reproducible after subsequent changes to the current modulebook, including the deletion of modules.

4.5.3. Supported Version-Control Operations

Table 4.2 shows the main operations supported by the version-control system.

Table 4.2.: Supported version-control operations.

User operation	**Backend behavior**	**Mainly affected tables**	**Frontend result**
Create module	The backend creates a `SOURCE_INITIAL_MODULE` change set, inserts the new module into `modules`, sets `created_change_set_id`, and writes one `module.created` record to `module_changes` with `old_value = null` and a compact module snapshot as `new_value`.	`modulebook_change_sets`, `modules`, `module_changes`	The module history shows that the module was created. The modulebook history shows the corresponding change set.

User operation	Backend behavior	Main affected tables	Frontend result
Edit module field	The frontend sends `changedFieldPaths`. The backend updates the current module state, creates a `SOURCE_MANUAL_EDIT` change set, and records only the fields that actually changed.	`modules`, `modulebook_change_sets`, `module_changes`	The module history shows old and new values. The modulebook history summarizes the changed modules and fields.
Change module status	The backend updates `modules.module_status`, creates a `SOURCE_MANUAL_EDIT` change set, stores the new status in the reason, and records only `module.moduleStatus`.	`modules`, `modulebook_change_sets`, `module_changes`	The history shows the status change as one version-control operation.
Delete module	The user must confirm the deletion by entering `DELETE`. The backend creates a `SOURCE_DELETE` change set, writes a `module.deleted` snapshot, and then physically deletes the module row.	`modulebook_change_sets`, `module_changes`, `modules`	The delete action remains visible in history, but this delete change set cannot be directly restored.
PDF import or reimport	The backend creates a `SOURCE_PDF_IMPORT` change set. Imported modules are matched by module id or module number, then created or updated. Existing modules that are not found in the import can be marked with `MISSING_FROM_IMPORT`.	`modulebook_change_sets`, `module_changes`, `modules`	The modulebook history shows the PDF import operation.
First PDF import	In addition to the normal PDF import behavior, the backend automatically creates an `INITIAL_IMPORT` modulebook version if this is the first PDF import of the modulebook. The default message is `Version 1`.	`modulebook_change_sets`, `module_changes`, `modules`, `modulebook_versions`	The initial version appears in the version list.
Save modulebook version	The backend stores the current modulebook and all active modules as a full JSON snapshot. The version number is generated by `max + 1`.	`modulebook_versions`	The saved version appears in the modulebook version list.
Delete modulebook version	The backend deletes the selected row from `modulebook_versions`. However, the `INITIAL_IMPORT` version cannot be deleted.	`modulebook_versions`	The version list is updated.
View modulebook version	The backend reads `snapshot_json`	`modulebook_versions`	The frontend opens the historical view with `versionId`.
Compare module changes	The backend reads `module_changes` for one module and aggregates old and new values within the selected change set range.	`module_changes`	The module comparison page shows field-level differences.
Compare modulebook versions	The backend combines change sets and saved versions as version points, then calculates differences from `module_changes` within the selected range.	`modulebook_change_sets`, `module_changes`, `modulebook_versions`	The modulebook comparison page shows changed modules and changed fields.

User operation	Backend behavior	Main affected tables	Frontend result
Restore before change set	The backend restores the state before a selected change set. It finds the target change set and all later active change sets, then writes fields back using the earliest old values. Later created modules may be deleted after explicit confirmation.	`modules`, `modulebook_change_sets`, `module_changes`	Later change sets are marked as discarded in history.
Force restore-before	If restore-before would delete modules created later, the first request returns a conflict. After user confirmation, the frontend sends `force=true`.	Same as restore-before	The frontend shows a second confirmation dialog.
Reset to initial import	The backend restores the state after the first `SOURCE_PDF_IMPORT`, deletes later change sets, and removes later modulebook versions while keeping the initial import version.	`modules`, `modulebook_change_sets`, `module_changes`, `modulebook_versions`	The modulebook returns to the initial imported state.
Restore modulebook version	The backend restores from `modulebook_versions.snapshot_json`. Existing modules are updated, missing modules are recreated, and active modules not in the snapshot are set to `DEACTIVE`.	`modules`, `modulebook_change_sets`, `module_changes`	The history shows that the modulebook was restored to a saved version.

4.6. FR3 – Module Handbook Import Implementation

The import pipeline converts existing module handbook PDFs into structured module data before the data is stored in PostgreSQL (Figure 4.5).

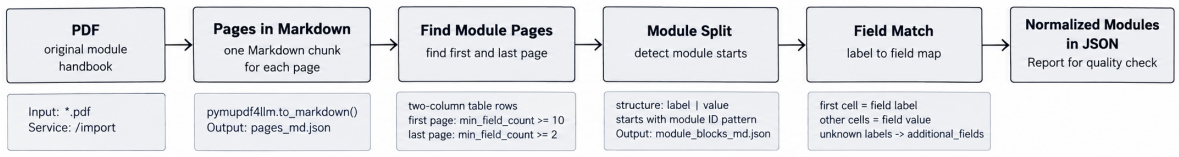


Figure 4.5.: PDF import pipeline from the source document to structured module data.

The PDF is processed by the Python/FastAPI import service, which converts the document into JSON, detects individual module blocks, and maps their content to structured module fields. The normalized module data is then sent to the Spring Boot application and stored in PostgreSQL.

Three main challenges are faced during this process.

Challenge 1: Find the modules. One challenge is detecting where the module is, specifically where a individual module begins and ends.

The pipeline first identifies which pages contain modules. In the original module handbook PDFs, modules are usually stored in tables. Therefore, the pipeline scans the extracted page-level Markdown and searches for recurring two-column table structures.

A page is treated as a possible module page when it contains table rows in the form `| label | value |` and when the labels in the left column match known module-related field labels. These known labels are defined in `base.yaml`, for example `Modulnummer`, `Titel`, `Leistungspunkte`, `Workload`, `Status`, `Inhalte`, and `Literatur`.

As the module pages are usually continuous, the pipeline looks for the first and the last module page. The first module page is detected with a stricter threshold: the page must contain at least **ten** matched module-related fields. The last module page is detected with a lower threshold of two matched fields, because the final page of a module description may contain only a small number of remaining fields, such as literature or additional notes. After the first and last module pages have been detected, only pages within this range are used for module splitting.

To detect the start of an individual module, the pipeline then uses two signals:

- the table row structure, usually in the form `| label | value |`;
- a module-number field, such as `Modulnummer`, `Module ID`, or `Modul ID`.

The module-number field is used as an anchor because it normally marks the beginning of a new module. The value next to this field is matched against predefined regular expressions, for example:

```
^\s*Modulnummer\s+([A-Z]{1,4}\d{1,2}(?:[.-]\d{1,2})?[a-z]?|[A-Z]{2,4})\b
^\s*Module\s+ID(?:\s+Modulkürzel)?\s+([A-Z]{1,4}\d{1,2}(?:[.-]\d{1,2})?[a-z]?|[A-Z]{2,4})\b
^\s*Modul\s*ID\s+([A-Z]{1,4}\d{1,2}(?:[.-]\d{1,2})?[a-z]?|[A-Z]{2,4})\b
```

When such a row is found, the pipeline starts a new module block. All following table rows are assigned to this module until the next valid module-number row is detected.

Challenge 2: Find the field. A second challenge is to find the field. Within each module, the pipeline detects field boundaries by checking the first cell of each table row. If the text in the first cell matches one of the predefined field labels, the row is treated as the start of a new field. The corresponding value is then extracted from the second cell or from the following content. However, different module handbooks may use different labels for the same semantic field. For example, “Module ID”, “Modulkürzel”, and “Modulnummer” are normalized to the same internal module identifier. The pipeline stores known label variants in the file `base.yaml` (Field mapping see table Table 3.1). The normalization step uses this mapping to translate extracted handbook labels into the internal field names. The import report supports this process by listing unmatched labels and missing required fields. If a handbook introduces a new label, the label can be added to the matching field entry in `base.yaml`. This keeps the label handling configurable and allows the pipeline to improve without code changes.

Challenge 3: Validation. The pipeline was tested on 5 NC Bachelor module handbooks. The number of detected modules were compared with the actual value.

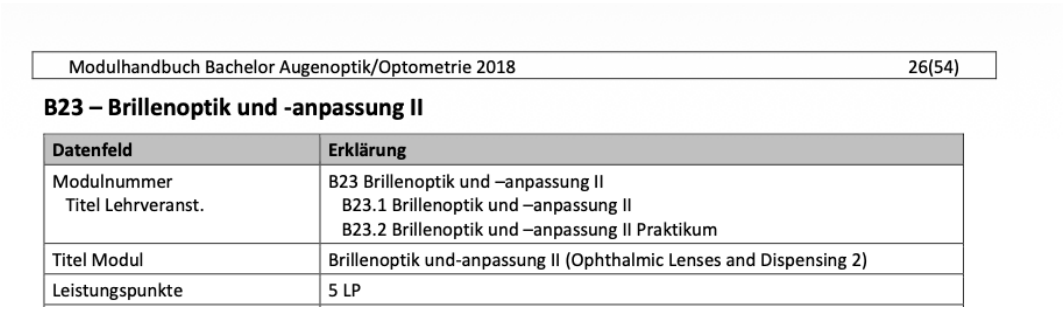
Module handbook	Actual modules	Detected modules
Architektur (B.Sc.)	39	39
Augenoptik (B.Sc.)	49	48
Data Engineering (B.Sc.)	38	38
Maschinenbau (B.Sc.)	85	85
Medieninformatik (B.Sc.)	54	54

Table 4.3.: Comparison between the actual number of modules and the number of modules detected by the import pipeline.

The results show high accuracy in module-level detection. Only one module was not detected correctly. The reason was an irregular title structure (see Figures 4.6 and 4.7).

B21	Binokularsehen II (Binocular Vision 2) B21.1 Binokularsehen II B21.2 Binokularsehen II Praktikum	5	ACTIVE	View Edit History Deactivate Delete
B22B23 Brillenoptik und –anpassung II B23.1 Brillenoptik und –anpassung II B23.2 Brillenoptik und –anpassung II Praktikum	Spezielle Optometrische Untersuchungen (Advanced Investigative Techniques in Optometry) B22.1 Spezielle Optometrische Untersuchungen B22.2 Spezielle Optometrische Untersuchungen PraktikumB23 Brillenoptik und –anpassung II B23.1 Brillenoptik und –anpassung II B23.2 Brillenoptik und –anpassung II PraktikumBrillenoptik und-anpassung II (Ophthalmic Lenses and Dispensing 2)	5	ACTIVE	View Edit History Deactivate Delete
B24	Low Vision II (Low Vision 2) B24.1 Low Vision II B24.2 Low Vision II Praktikum	5	ACTIVE	View Edit History Deactivate Delete
B25	Klinische Optometrie I (Clinical Optometry 1) B25.1 Klinische Optometrie B25.2 Klinische Optometrie Praktikum	5	ACTIVE	View Edit History Deactivate Delete

Figure 4.6.: Import result: the module number `B23` was failed to be detected



Modulhandbuch Bachelor Augenoptik/Optometrie 2018 26(54)

B23 – Brillenoptik und -anpassung II

Datenfeld	Erklärung
Modulnummer Titel Lehrveranst.	B23 Brillenoptik und –anpassung II B23.1 Brillenoptik und –anpassung II B23.2 Brillenoptik und –anpassung II Praktikum
Titel Modul	Brillenoptik und-anpassung II (Ophthalmic Lenses and Dispensing 2)
Leistungspunkte	5 LP

Figure 4.7.: Import issue caused by an irregular table structure: the module number `B23` was merged into title content, so the pipeline failed to detect the start of a new module.

Field-level validation has not yet been implemented systematically. At the current stage, the pipeline only reports potentially problematic fields, for example fields that are empty after extraction. This report helps with manual review, but it does not yet prove that all extracted field values are correct.

A key difficulty is the lack of reliable ground truth data. For a reliable validation, the expected number of modules and the expected content of each field need to be available.

Alternative AI-Based Extraction Approach

During the early development of the PDF import pipeline, AI-based document extraction was investigated as an alternative to the rule-based approach. Research on document understanding has shown that both textual content and two-dimensional document structure are important for information extraction. Earlier approaches such as Chargrid Katti et al., 2018 explicitly preserve document layout, while later models such as LayoutLM Xu et al., 2020, Donut Kim et al., 2022, and DocLLM Wang et al., 2024 apply Transformer-based and large-language-model approaches to structured document understanding.

Based on these approaches, several LLM-based extraction strategies were evaluated during early prototyping using the BHT RIS LLM API with the model `bht/large(cyankiwi/MiniMax-M2.7-AWQ-4bit)`. The experiments examined whether an LLM-based approach could extract information from module handbook PDFs and map it into a structured module and field schema.

The first approach used regular expressions to split the PDF into pages in json and then asked the LLM to identify where individual modules started and ended. While processing page-level JSON directly in ChatGPT (GPT-5.5) was fast, the same task through the LLM API was too slow, processing one module took approximately two minutes. Therefor this approach is unsuitable for complete module handbooks containing many modules.

The second approach used the LLM for field extraction after the module boundaries had already been detected. In this setup, each module block was passed to the model with a predefined JSON schema, asking it to extract fields such as module number, title, credits, workload, learning objectives, examination form, contents, literature, and source pages. In the ChatGPT interface, one PDF could be processed in about two to three minutes. However, when using the BHT RIS LLM API with `bht/large`, the field extraction step was much slower and could exceed 20 minutes before timing out. The results also depended strongly on the prompt. Without a detailed schema, the model often returned only a few fields, usually around three, which was not sufficient for reliable import.

The third approach attempted to process the entire PDF in one request. The PDF text was extracted page by page, combined into one large prompt, and sent to the LLM with instructions to return all modules in a fixed JSON structure. In ChatGPT, one PDF could be processed in approximately four minutes when a schema was provided. However, the output was not reliable: in some cases, the model returned only four fields instead of the full schema. The API-based version was also very slow and prone to timeouts. The author also tested a direct-PDF approach, where the PDF file itself was sent to the API as a file payload, but the BHT RIS endpoint returned `501 Unknown part type:`

`file`, indicating that this endpoint did not support direct PDF/file inputs.

Overall, the LLM-based approaches were useful for experimentation, but they were not suitable for a quick and reliable batch import pipeline at that time (06.2026). The main limitations were long API response times, timeout problems, incomplete field extraction, and lack of support for direct PDF input. For this reason, the current pipeline was implemented as a rule-based extraction process using `pymupdf4llm`, Markdown table parsing, regular expressions, and a configurable field mapping in `base.yaml`. However, the current pipeline does not yet include a systematic validation process, and the rule-based approach has known limitations. As discussed above, the table structure in the PDFs is not always stable or consistent. Some tables span multiple pages. If a module table does not end on one page and continues on the next page, parts of the content may be missed or assigned incorrectly because the continuation page may not contain a repeated field label. LLM may be used as an validation layer in the future work.

4.7. FR4 – Module Handbook Generation and Export Implementation

The module handbook is generated from structured application data. For the current modulebook, the application loads the active modules from the database and renders them as a print-ready HTML document (FR4.1).

For a saved modulebook version, the application loads the historical module data from `modulebook_versions. snapshot_json`.

In both cases, the backend renders a print-ready HTML representation, and the browser's print function is used to print or export the module handbook as a PDF.

4.8. Validation, Error Handling, and Testing

Validation is implemented at both the frontend and backend levels. The frontend prevents common invalid input through HTML form rules, such as required fields, maximum lengths, and non-negative numeric values. The backend additionally applies DTO validation and service-level checks to reject clearly invalid or system-breaking input.

Table 4.4 summarizes selected validation rules in the current implementation.

Table 4.4.: Examples of frontend and backend validation rules.

Field / Operation	**Validation Rule**
Module number	Required and limited to 50 characters.
Title	Required and limited to 255 characters.
Credits	Required and cannot be negative.
Module update	Requires a commit message.
Module status	Must be either `ACTIVE` or `DEACTIVE`.
Modulebook degree type	Must be either `BACHELOR` or `MASTER`.
PDF import	An imported module must contain a module number, title, and credits.

Error handling is implemented through service-level exceptions. For example, the backend returns a `400 Bad Request` response for missing required values, invalid enum-like values, empty PDF files, unsupported file types, or missing import data.

Testing is implemented through unit and controller tests for the core parts of the system. The current tests cover module services, module controllers, modulebook controllers, PDF import services, imported-module mapping, PDF runner behavior, and change-log logic.

4.9. CI/CD, Deployment, and Observability

GitLab CI/CD is used to test, build, and deploy the system (Fig. 4.8). The CI/CD pipeline consists of three stages:

- **test**: runs the Maven test suite with a temporary PostgreSQL database;
- **docker**: builds and pushes the Docker images; and
- **deploy**: deploys the images to the BHT Kubernetes cluster.

The Spring Boot backend is packaged as a Docker image and deployed as a Pod in the BHT Kubernetes cluster. The deployed application is available at http://modulebook.project.ris.bht-berlin.de/index.html. The Python PDF import service is packaged as a separate Docker image and runs in a separate Pod. The Spring Boot application communicates with the Python service through the internal Kubernetes Service `pdf-import-service:8000`. PostgreSQL is deployed inside the cluster with persistent storage.

Basic technical observability is implemented using OpenTelemetry according to NFR4. The Spring Boot service is instrumented automatically through the OpenTelemetry

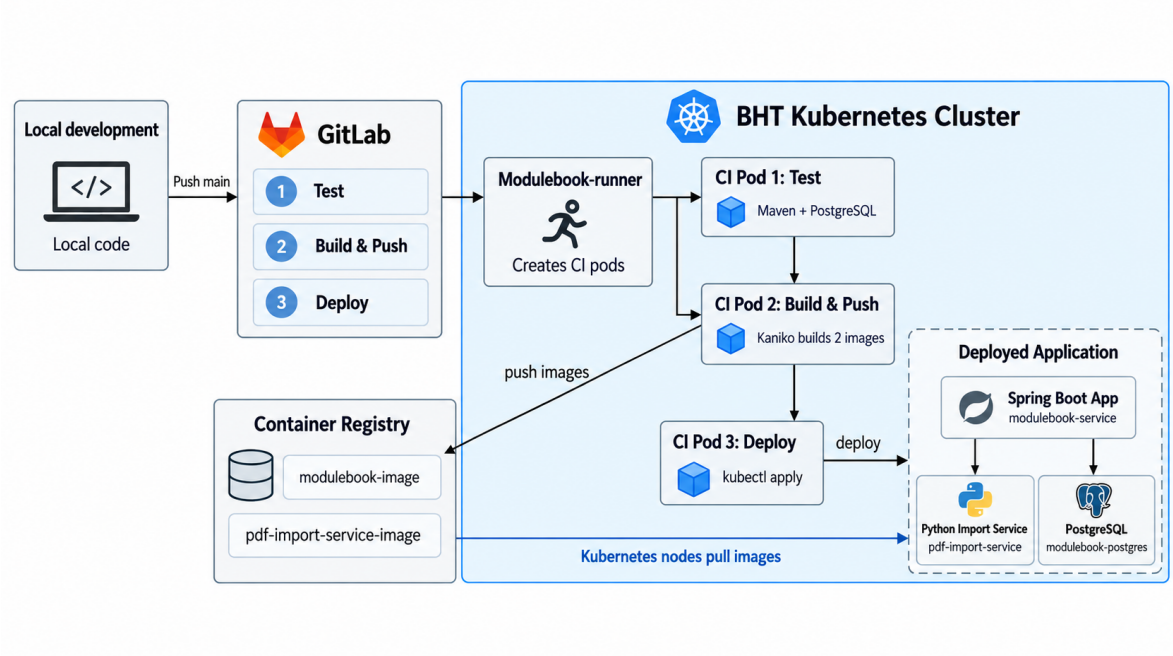


Figure 4.8.: GitLab CI/CD pipeline overview.

Java Agent, which collects traces for HTTP requests, Spring MVC operations, and JDBC/PostgreSQL access.

Trace data is exported via OTLP gRPC to the OpenTelemetry Collector in the Kubernetes cluster and is subsequently available for inspection in OpenObserve. The Python PDF import service also uses OpenTelemetry instrumentation for FastAPI requests.

5. Evaluation

The requirements-based evaluation shows that the core MVP functions were implemented (Table 5.1). Most technical non-functional requirements were also addressed (Table 5.2). However, usability of the final implementation was not formally evaluated, and several requirements identified during the elicitation process remain outside the implemented MVP scope (Table 5.3).

5.1. Evaluation against MVP Functional Requirements

Table 5.1.: Evaluation against MVP functional requirements.

ID	Requirement	Result
FR1 – Module Handbook Management		
FR1.1	Structured module data	Implemented
FR1.2	View, create, and delete module handbooks	Implemented
FR1.3	View, create, update, and delete modules	Implemented
FR1.4	Input assistance and validation	Implemented
FR1.5	Module status management	Implemented
FR2 – Version Control		
FR2.1	Change traceability	Implemented
FR2.2	Change sets	Implemented
FR2.3	Save module handbook version	Implemented
FR2.4	Version history	Implemented
FR2.5	Version comparison	Implemented
FR2.6	Restore previous state	Implemented
FR3 – Module Handbook Import		
FR3.1	PDF import	Implemented

ID	Requirement	Result
FR3.2	Module and field detection	Implemented
FR3.3	Initial imported version	Implemented
FR3.4	Re-import and update	Implemented
FR4 – Module Handbook Generation and Export		
FR4.1	Generate module handbook view	Implemented
FR4.2	Export current module handbook	Implemented
FR4.3	Export saved module handbook version	Implemented

5.2. Evaluation against Non-functional Requirements

Table 5.2.: Evaluation against non-functional requirements.

ID	Requirement	Result	Evidence
NFR1	Usability	Partially evaluated	User requirements and prototype feedback were incorporated, but no final usability study was conducted.
NFR2	Maintainability	Implemented at architectural level	Layered architecture, separate services/repositories, Flyway migrations.
NFR3	Deployability / Portability	Implemented	Application deployed in the BHT Kubernetes environment.
NFR4	Observability and Technical Traceability	Implemented at basic level	OpenTelemetry traces from the Spring Boot and PDF import services are exported through the cluster collector and can be inspected in OpenObserve.

5.3. Requirements Outside the MVP Scope

Table 5.3.: Requirements outside the MVP scope.

Requirement identified during elicitation	**Result / Scope**
Authentication / Login	Not implemented – outside MVP scope.
Role-based access control	Not implemented – outside MVP scope.
Full approval workflow	Not implemented – outside MVP scope.
Collaborative editing	Not implemented – outside MVP scope.

6. Conclusion

This thesis addressed the problem of managing module handbooks as manually maintained documents. Although module handbooks are published as Word or PDF files, their underlying information is largely structured and can therefore be represented and managed as application data.

Based on user requirements and iterative prototype development, a web-based MVP was designed and implemented. The system supports structured module and modulebook management, change tracking and saved modulebook versions, import of existing module handbook PDFs, and generation of printable module handbooks. The implementation uses Spring Boot and PostgreSQL as the core application backend, together with a separate Python-based PDF import service.

The work shows that a web-based system for managing module handbooks can be designed by translating the document-based workflow into structured domain concepts and refining these concepts iteratively with users. Three design decisions were particularly important: separating structured module data from document presentation, separating the current state from detailed change tracking and saved modulebook versions, and supporting the migration of existing PDF documents into the structured system.

A formal usability evaluation of the final MVP was not conducted within the scope of this thesis. User feedback was collected during requirements elicitation and prototype refinement, but the implemented system itself still requires evaluation with its intended users.

Future work should evaluate the implemented system with its intended users and extend the MVP with authentication, role-based permissions, and an approval workflow. The PDF import process could also be further improved to support validated extraction of a broader range of existing module handbooks.

References

Katti, A. R., Reisswig, C., Guder, C., Brarda, S., Bickel, S., Hohne, J., & Faddoul, J. B. (2018). Chargrid: Towards understanding 2d documents. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.

Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). Ocr-free document understanding transformer. *European Conference on Computer Vision*.

Wang, D., Raman, N., Sibue, M., Ma, Z., Babkin, P., Kaur, S., Pei, Y., Nourbakhsh, A., & Liu, X. (2024). Docllm: A layout-aware generative language model for multimodal document understanding. *arXiv preprint arXiv:2401.00908*.

Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). Layoutlm: Pre-training of text and layout for document image understanding. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.

A. Supplementary Material

A.1. Links

- **Modulebook (Project):** modulebook.project.ris.bht-berlin.de
- **Modulebook (Source code):** gitlab.bht-berlin.de/ris/modulebook
- **Clickable Figma prototype:** Interactive Module Editing Prototype
- **Core function design (Figma):** Modulebook Core Function Design

A.2. Initial Requirements Meeting Protocol

Meeting Protocol, Referat Studium und Lehre, 25 June

1. **Felder**
   a) Es ist wichtig, alle relevanten Felder zu berücksichtigen und diese möglichst genau zu definieren, z. B. durch getrennte Angaben für Präsenzzeit und Selbststudiumszeit.
   b) Nicht alle Felder sind Pflichtfelder, z. B. Literatur.
   c) Alle Felder aus der Anlage "Studienplan" sind Pflichtfelder.
   d) Felder können bei Bedarf hinzugefügt werden.
   e) Es muss Validierungsregeln für Felder geben, damit klar ist, welche Eingaben zulässig sind, z. B. nur Zahlen bis zu einem bestimmten Maximalwert.
   f) Es wäre hilfreich, Eingabehilfen bzw. Eingabebeispiele anzubieten (? Taste), damit Benutzer*innen wissen, was wo eingetragen werden soll.
   g) Später könnte man prüfen, ob zwei Varianten des Modulhandbuchs generiert werden können: eine vollständige Version mit allen Feldern und eine reduzierte Version mit weniger Feldern, um geringfügige Anpassungen der SPO möglichst zu vermeiden.
2. **Benutzer*innen**
   a) Es gibt unterschiedliche Benutzer*innen mit unterschiedlichen Rollen.
   b) Benutzer*innen haben unterschiedliche Rechte, z. B. kein Zugriff, Lesen oder Schreiben.
3. **Bearbeitung**

a) Es muss möglich sein, Dokumente kollaborativ zu bearbeiten.

b) Es muss möglich sein, ein Dokument an eine andere Benutzerin oder einen anderen Benutzer zur Bearbeitung weiterzugeben.

c) Es muss möglich sein, Änderungen im Dokument nachzuvollziehen.

d) Es muss sowohl Deutsch als auch Englisch unterstützt werden.

A.3. User Meeting: Figma Test Protocol

Meeting, 14–15 May

Klickbarer Prototyp: siehe Section A.1.

Wünsche und Plan für die nächste Version des klickbaren Prototyps

Auf der Modulübersichtsseite

- **LP-Summe in der Endzeile anzeigen**

  1 Semester = 30 LP (Bologna-Standard).

  Bachelor mit 6 Semestern = 180 LP gesamt.

  Eine automatische Prüfung ist gewünscht.

- **Präsenzstunden und Selbststudiumsstunden pro Modul anzeigen**

  Zusätzlich soll in der Endzeile jeweils die Summe angezeigt werden.

- **Spaltenreihenfolge anpassen**

  Modul, Modulname, Modultyp, Semester, LP, Präsenzstunden, Selbststudiumsstunden, SU, Ü, Modulverantwortung.

Auf der Seite "Modul bearbeiten"

- **Modulkürzel soll editierbar sein**
- **Workload automatisch prüfen**
  - 1 Leistungspunkt (LP) = 30 Stunden.
  - Gesetzlicher Rahmen: 25–30 Stunden pro LP (BHT hält derzeit 30 fest, Abweichungen können zu Kritikpunkten führen).
- **SWS-Berechnung klar abbilden**
  - 1 SWS = 45 Minuten.
  - Berechnungsgrundlage: 17 Wochen pro Semester.
  - SWS ist die führende Eingabe.
  - Präsenzstunden werden daraus abgeleitet.

  - Selbststudium = Gesamt-LP-Stunden minus Präsenzstunden.

- **Warnhinweis bei Überschreitung der Gesamtstunden**
  - Die Eingabe soll die hinterlegte Gesamtstundenzahl nicht überschreiten.
  - Bei Überschreitung: Fehlermeldung oder Warnhinweis, aber kein hartes Blockieren.

- **Qualifikationsziele / Kompetenzen**
  - Gesamtzeichenzahl begrenzen.
  - Hinweis-Button mit Verlinkung einbauen (Bachelor, Master unterschied).
  - Links:
    * DQR: DQR-Niveaus
    * Qualifikationsrahmen HQR: Qualifikationsrahmen HQR
  - Optional: KI-Prompt zur automatischen Überprüfung.

- **Voraussetzungen**
  - Andere Module mit @ referenzierbar machen.
  - Zusätzlich Freitext ermöglichen.

- **Niveaustufe**
  - Auswahlbereich je nach Studiengang anpassen, z. B. für Master: 1.–4. Semester.

- **Lehr- und Lernform**
  - Hinweis-Button mit Informationen zur Kapazität verlinken (KapVO, K12).

- **Prüfungsform**
  - Fester Einleitungstext am Anfang:

    "Die Prüfungsform wird nach §19 (2) RSPO ..."

- **Unit-Beurteilung / Gewichtung**
  - Feld Unit-Beurteilung anzeigen.
  - Feld Unit-Gewicht anzeigen (automatische Prüfung abhängig von der Unit-Beurteilung S/U/I).
  - Feld Modulgewicht anzeigen, z. B. 5 / 175.

- **Inhalte**
  - Zeichenzahl begrenzen.

- **Literatur**
  - Musterliteratur über einen Hinweis-Button anzeigen.

- **Weitere Hinweise**

  - Vorgegebene Optionen plus Freitext.

- **Offene Punkte**
  - Verwendbarkeit.
  - Lerngebiet.
  - Prüfungsgruppen.

Studienplan-Anlage automatisch generieren

- Tabelle 1: alle Pflichtmodule mit Wahlpflicht-Platzhalter.
- Tabelle 2: Wahlpflichtmodule.

Module Handbook Version Control

- **Zwei Versionen vorsehen**
  - **Amtliche Mitteilung**

    Aktuell veröffentlichte Version, enthält nur aktive Module.
  - **Arbeitsversion**

    Entwurf für die nächste amtliche Mitteilung, enthält aktive, inaktive und gelöschte bzw. archivierte Module.
  - Unterschiedliche Rollen sollen prüfen und kommentieren können.
- **Änderungsprotokoll erweitern**

  Wunsch: Autor und Zeitstempel bei jeder Änderung anzeigen.

Neue Seiten

- **Neue Seite: Modul hinzufügen**
- **Neue Seite: Felder verwalten**
  - Bestehende Felder bearbeiten, z. B. Kategorien für feste Optionen ergänzen oder Feldformat ändern.
  - Neue Felder hinzufügen.
  - Neue Felder gelten für alle Module, aber erst ab der aktuellen Arbeitsversion.

Weitere Wünsche

- **Kompakte Druckansicht**

  Kompaktes Layout für den Ausdruck, alle Inhalte auf einer Seite.
- **Mehrere Module gleichzeitig bearbeiten**

  z. B. “Weitere Hinweise” für mehrere Module auf einmal ändern.